



CPCS 211: Digital Logic Design

Chapter2 : Compinational Systems

Outline

- Combinational Systems
- Design Process for Combinational Systems
- Don't Care Conditions
- Development of Truth Tables
- Switching Algebra
- Manipulation of Algebraic Functions
- Implementation of Functions with AND, OR and NOT gates
- Complements
- NAND and NOR Gates and Exclusive-OR
- Manipulation of Algebraic Functions and NAND Gate Implementations

Combinational systems

Definition of Combinational System:

In a combinational system, the output of any gate can be determined at any stage in terms of its inputs.

Design Process for Combinational Systems

A typical design process for a combinational system contains the following steps:

- **Problem statement** - a verbal description of the intended system
- **Develop a block diagram** of that system, utilizing available components and meeting the design objectives and constraints

Design Process for Combinational Systems

Design process can be further broken down into the following steps:

1. Represent each of the inputs and outputs in binary
 - If necessary, break the larger problem into smaller subproblems
 - For example, represent On with 1, Off with 0, or Up with 1 and Down with 0
 - Also, it is often most economical to take advantage of subsystem that already have been implemented, e.g. buy a 4-bit adder
2. Formalize the design specification either in the form of a truth table or of an algebraic expression
3. Simplify the description, e.g. using K-Maps
4. Implement the system with available components, subject to the design objectives and constraints

Don't Care Conditions

- In some systems, value of output is specified for only some of input conditions; referred to as *incompletely specified* functions
- For remaining input combinations, it does not matter what the output is, that is, we *don't care*
 - In a truth table, don't cares are indicated by an **X**
- When we design a system with don't cares, we may make the output either 0 or 1 for each don't care input combination
- If there are several don't cares, number of acceptable solutions greatly increases, since each don't care can be either 0 or 1, independently

Example

Table states that f must be

- 0 when a and b are 0
- 1 when a 0 and b 1 or when a 1 and b 0

it does not matter what f is, when a and b are both 1
that means that we can implement either f_1 or f_2
In other words, either f_1 or f_2 are acceptable

Table 2.3 Acceptable truth tables.

a	b	f_1	f_2
0	0	0	0
0	1	1	1
1	0	1	1
1	1	0	1

Table 2.2 A truth table with a don't care.

a	b	f
0	0	0
0	1	1
1	0	1
1	1	X

Real System Example

- In real systems, don't cares occur in several ways
- There may be some input combinations that never occur.
 - For example, if input is written in binary code for a decimal digit; there are only 10 possible input combinations (i.e. for digits from 0 to 9).
 - If a 4-bit code is used, then six of the input combinations never occur – i.e. 1010, 1011, 1100, 1101, 1110 and 1111
- When we build a system, we can design it such that the outputs would be either 0 or 1 for each of these don't care combinations, since that input never happens

Development of Truth Tables

- In digital system, each of inputs only takes on one of two values (0 or 1)
- Thus, if we have n inputs, there are 2^n input combinations and thus the
- truth table has 2^n rows
- These rows are normally written in the binary order of the inputs (if, for no other reason, than to make sure that we do not leave any out)
- The truth table has two sets of columns:
 - n input columns, one for each input variable, and
 - m output columns, one for each of the m outputs

Example

Truth table for OR gate is shown as follows:

Table 2.1 A two-input truth table.

<i>A</i>	<i>B</i>	<i>Y</i>
0	0	0
0	1	1
1	0	1
1	1	1

Example

- Truth table for a system with four inputs, **A**, **B**, **C**, and **D**, and one output, **Z**, such that **Z = 1** iff three of inputs are 1, as shown
- Problem statement requires to have only 1 output, but table contains 3 outputs
 - Reason is that we can have three different interpretations of the problem statement
- No controversy on behavior of system for first 15 rows of table, only question in completing table is in relation to last row

Table 2.4 Truth table for CE1.

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>Z</i> ₁	<i>Z</i> ₂	<i>Z</i> ₃
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
0	1	1	1	1	1	1
1	0	0	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	0	0	0
1	0	1	1	1	1	1
1	1	0	0	0	0	0
1	1	0	1	1	1	1
1	1	1	0	1	1	1
1	1	1	1	0	1	X

Example (Cont.)

- Does “3 of the inputs are 1” mean exactly three or does it mean at least 3?
 - If the former is true, then the last line of the truth table is 0, as shown for Z1
 - If the latter is true, then the last line of the table is 1, as shown in Z2
- Two other options, both shown as Z3:
 - We know that all four inputs will not be 1 simultaneously, and
 - We do not care what output is if all four inputs are 1; in those cases, last entry is X

Table 2.4 Truth table for CE1.

A	B	C	D	Z ₁	Z ₂	Z ₃
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
0	1	1	1	1	1	1
1	0	0	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	0	0	0
1	0	1	1	1	1	1
1	1	0	0	0	0	0
1	1	0	1	1	1	1
1	1	1	0	1	1	1
1	1	1	1	0	1	X

Another Example

- Consider a system in which a single light (that can be on or off) that can be controlled by any one of three switches. One switch is the **master on/off** switch. If it is down, the light is off. When the master switch is up, a change in the position of one of the other switches (from up to down or from down to up) will cause the light to change state
- Truth table for such a system is as follows:

Example cont.

- Switches are labeled: **a**, **b**, **c** (a is master switch)
- Use **1** for **up**, and **0** for **down**
- Light output is labeled **f**
 - 1 on **f** means that light is on
 - When a = 0, light is off (0), no matter what value of b and c
 - For input 100, two cases of light: f = 0 or 1

abc:

100 -> 10**1** -> 100

100 -> 10**1** -> 1**1**1

100 → 1**1**0 -> 100

100 -> 1**1**0 -> 1**1**1

Table 2.5 Truth tables for CE2.

<i>a</i>	<i>b</i>	<i>c</i>	<i>f</i>
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

(a)

<i>a</i>	<i>b</i>	<i>c</i>	<i>f</i>
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

(b)

Example cont.

abc:

100 → 10**1** → 100

100 → 10**1** → 111

100 → 1**1**0 → 100

100 → 1**1**0 → 111

Assume that switches *b* and *c* in down position cause light to be off, **f = 0**

- 5th row (**100**) will have output 0
- When one of these switches is up (**101**, **110**), then light must be on
- From either of these states, changing *b* or *c* will either return system to the **100** input state or move it to state **111**; output is 0

Table 2.5 Truth tables for CE2.

<i>a</i>	<i>b</i>	<i>c</i>	<i>f</i>
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

(a)

<i>a</i>	<i>b</i>	<i>c</i>	<i>f</i>
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

(b)

Example cont.

abc:

100 → 101 → 100

100 → 101 → 111

100 → 110 → 100

100 → 110 → 111

abc:

111 → 101 → 100

111 → 101 → 111

111 → 110 → 100

111 → 110 → 111

We could have started with some other fixed value, such as

- switches *b* and *c* up means that the light is on, **f = 1**
- switches *b* and *c* down means that the light is on, **f = 1**
- Either of these would produce TT of Table 2.5b, which is equally acceptable

Table 2.5 Truth tables for CE2.

<i>a</i>	<i>b</i>	<i>c</i>	<i>f</i>
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

(a)

<i>a</i>	<i>b</i>	<i>c</i>	<i>f</i>
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

(b)

Another Example

The truth table for 1-bit binary full adder is as follows

- The system has 3 inputs (2 bits to be added plus carry from next lower order bit) and produces 2 outputs: a sum bit and a carry to the next higher order position

Table 1.5 One-bit adder.

a	b	c_{in}	c_{out}	s
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

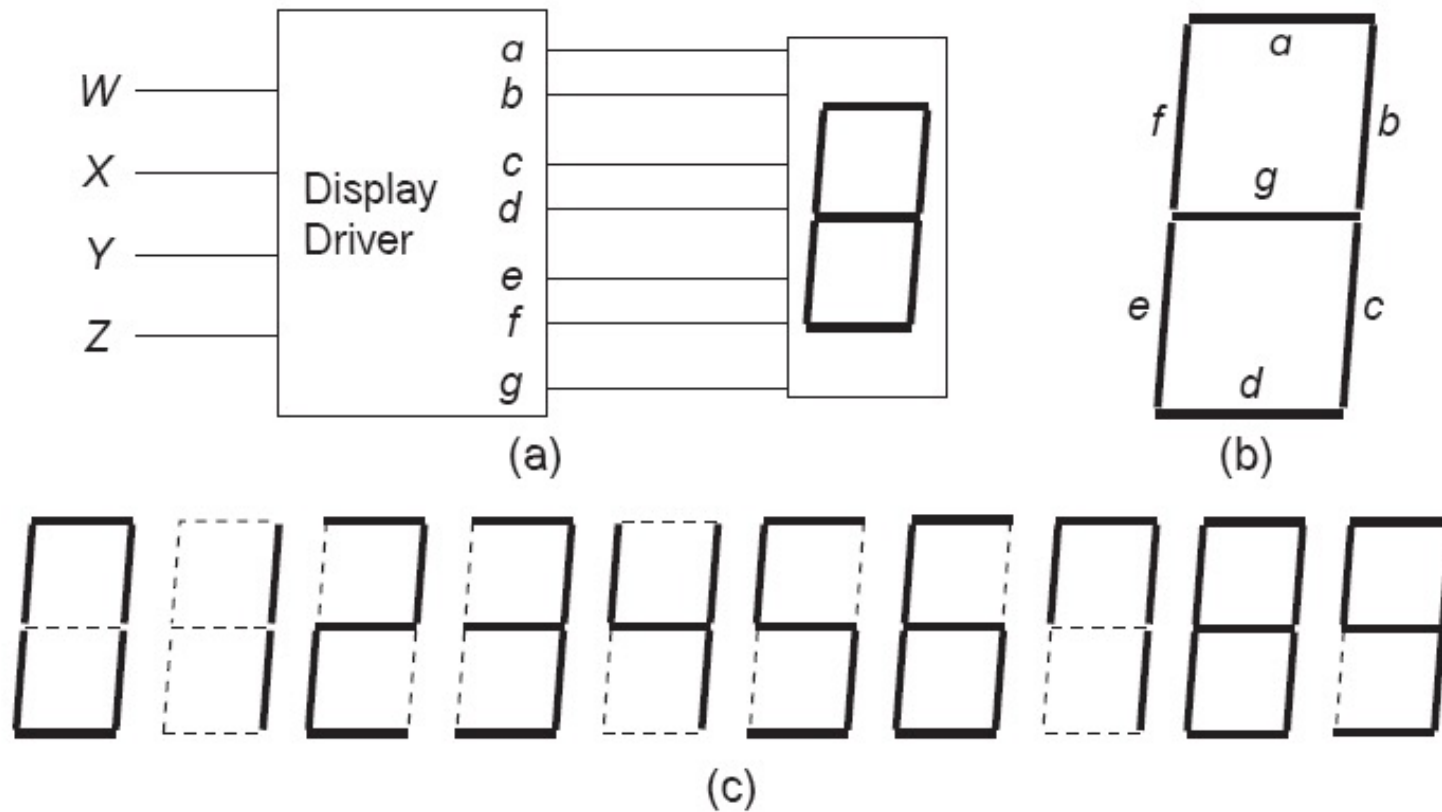
Display Driver Example

A system that has as its input the code for a decimal digit and produces as its output the signals to drive a seven-segment display, such as those on most digital watches and numeric displays

- Inputs are BCD code (i.e. 8421 Code) for decimal digit; labeled W, X, Y, Z
- Display driver must provide seven inputs to the display: a, b, c, d, e, f, and g
- First 10 combinations of input variables are used for BCD codes of digits 0 to 9
- Last 6 bit combinations are used as don't care conditions

Display Driver cont.

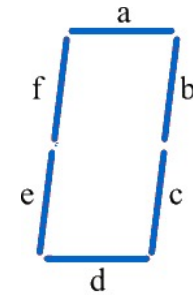
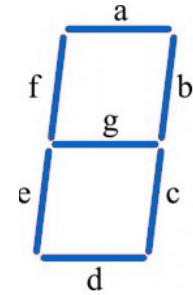
Figure 2.3 A seven-segment display.



cont.

Table 2.6 A truth table for the seven-segment display driver.

Digit	<i>W</i>	<i>X</i>	<i>Y</i>	<i>Z</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	X	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	X	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	X	0	1	1
–	1	0	1	0	X	X	X	X	X	X	X
–	1	0	1	1	X	X	X	X	X	X	X
–	1	1	0	0	X	X	X	X	X	X	X
–	1	1	0	1	X	X	X	X	X	X	X
–	1	1	1	0	X	X	X	X	X	X	X
–	1	1	1	1	X	X	X	X	X	X	X



One More Example

Consider a system with 3 inputs, a , b , and c , and 4 outputs, w , x , y , z

- Output is a binary number equal to the largest integer that meets the input conditions:
 - $a = 0$ means odd, $a = 1$ means even
 - $b = 0$ means prime, $b = 1$ means not prime
 - $c = 0$ means < 8 , $c = 1$ means ≥ 8
- Some inputs may never occur, the output is never all 0's.

Example cont.

a = 0 odd a = 1 even
 b = 0 prime b = 1 not prime
 c = 0 < 8 c = 1 ≥ 8

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

	a	b	c	w	x	y	z	largest	
odd, prime, < 8	0	0	0	0	1	1	1	7	1, 3, 5, 7
odd, prime, ≥ 8	0	0	1	1	1	0	1	13	11, 13
odd, not prime, < 8	0	1	0	X	X	X	X		X
	0	1	1	1	1	1	1		
	1	0	0	0	0	1	0		
	1	0	1	X	X	X	X		
	1	1	0	0	1	1	0		
	1	1	1	1	1	1	0		

Switching Algebra

- Need algebraic description to analyze and design systems
- Need algebra for several reasons; the most obvious is that with a network of gates, we need to obtain a specification of the output in terms of the input
- In the design process, often obtain an algebraic expression much more complex than is necessary
- Algebra simplifies that expression
 - minimize the amount of logic (gates) needed to implement it

Definition of Switching Algebra

- Switching algebra is **binary**, that is, all variables and constants take on one of two values: **0** and **1**
- We will define the three operators of switching algebra and then develop a number of properties of switching algebra

Operators

1. OR: written as $+$, i.e. $a + b$ (read as a OR b)
is 1 iff $a = 1$ or $b = 1$ or both

2. AND: written as $.$ or simply two variables catenated, i.e. $a . b$ or ab
(read as a AND b)
is 1 iff $a = 1$ and $b = 1$

3. NOT: (written as $'$), i.e. a' (read as NOT a)

- is 1 iff $a = 0$
- The term **complement** is sometimes used instead of NOT
- Operation also referred to as **inversion**, and device (gate) is called **inverter**

Gates

- Symbols (gates) for implementing previous operations are:

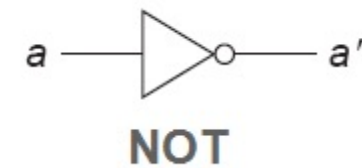
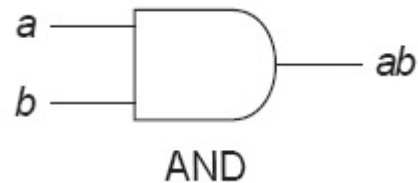
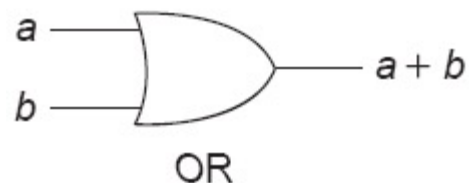
Table 2.7 Truth tables for OR, AND, and NOT.

a	b	$a + b$
0	0	0
0	1	1
1	0	1
1	1	1

a	b	ab
0	0	0
0	1	0
1	0	0
1	1	1

a	a'
0	1
1	0

- A gate is a basic circuit element with one output that implements one of the basic functions such as OR, AND and NOT



Properties of Switching Algebra

$$P1a. \quad a + b = b + a$$

$$P1b. \quad ab = ba$$

commutative

$$P2a. \quad a + (b + c) = (a + b) + c$$

$$P2b. \quad a(bc) = (ab)c$$

associative

$$P3a. \quad a + 0 = a$$

$$P3b. \quad a \cdot 1 = a$$

identity

$$P4a. \quad a + 1 = 1$$

$$P4b. \quad a \cdot 0 = 0$$

null

$$P5a. \quad a + a' = 1$$

$$P5b. \quad a \cdot a' = 0$$

complement

$$P3aa. \quad 0 + a = a$$

$$P3bb. \quad 1 \cdot a = a$$

$$P4aa. \quad 1 + a = 1$$

$$P4bb. \quad 0 \cdot a = 0$$

$$P5aa. \quad a' + a = 1$$

$$P5bb. \quad a' \cdot a = 0$$

$$P6a. \quad a + a = a$$

$$P6b. \quad a \cdot a = a$$

idempotency

$$P7. \quad (a')' = a$$

involution

$$P8a. \quad a(b + c) = ab + ac$$

$$P8b. \quad a + bc = (a + b)(a + c)$$

distributive

Note:

- Every (b) part is the dual of its corresponding (a) part.
- To obtain the dual of a property, interchange OR and AND and the constants 0 and 1.

Proving the Above Properties

Can use truth table to prove each property, e.g., TT to prove Prop. 8b:

$$\text{P8b. } a + bc = (a + b)(a + c)$$

Table 2.8 Truth table to prove Property 8b.

<i>a</i>	<i>b</i>	<i>c</i>	<i>bc</i>	LHS	<i>a + b</i>	<i>a + c</i>	RHS
0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	1	1	1
1	0	1	0	1	1	1	1
1	1	0	0	1	1	1	1
1	1	1	1	1	1	1	1

Example

$$f = y'z' + x'y + x'yz'$$

$$g = xy' + x'z' + x'y$$

$$h = (x' + y')(x + y + z')$$

- Construct a truth table and show which of the three functions are equal

- The truth table for each of the above Boolean functions is shown:

$x y z$	$y'z'$	$x'y$	$x'yz'$	f	xy'	$x'z'$	$x'y$	g	$x' + y'$	$x + y + z'$	h
0 0 0	1	0	0	1	0	1	0	1	1	1	1
0 0 1	0	0	0	0	0	0	0	0	1	0	0
0 1 0	0	1	1	1	0	1	1	1	1	1	1
0 1 1	0	1	0	1	0	0	1	1	1	1	1
1 0 0	1	0	0	1	1	0	0	1	1	1	1
1 0 1	0	0	0	0	1	0	0	1	1	1	1
1 1 0	0	0	0	0	0	0	0	0	0	1	0
1 1 1	0	0	0	0	0	0	0	0	0	1	0

Example cont.

$$f = y'z' + x'y + x'yz'$$

$$g = xy' + x'z' + x'y$$

$$h = (x' + y')(x + y + z')$$

- From this table, we can conclude that for input combination 101, $f = 0$, but $g = h = 1$

- Thus, f is not equal to either of the other functions.

- But columns for g and h are identical; thus, $g = h$

$x y z$	$y'z'$	$x'y$	$x'yz'$	f	xy'	$x'z'$	$x'y$	g	$x' + y'$	$x + y + z'$	h
0 0 0	1	0	0	1	0	1	0	1	1	1	1
0 0 1	0	0	0	0	0	0	0	0	1	0	0
0 1 0	0	1	1	1	0	1	1	1	1	1	1
0 1 1	0	1	0	1	0	0	1	1	1	1	1
1 0 0	1	0	0	1	1	0	0	1	1	1	1
1 0 1	0	0	0	0	1	0	0	1	1	1	1
1 1 0	0	0	0	0	0	0	0	0	0	1	0
1 1 1	0	0	0	0	0	0	0	0	0	1	0

Algebraic Functions

- Algebraic functions or expression involve:
- **Variables** either
 - complemented or
 - uncomplemented
- **Operators**
 - $+$ (OR) and
 - $.$ (AND)

literal

- A **literal** is the appearance of a variable or its complement
 - e.g. a and b' are literals
- In determining the complexity of an expression, one of the measures is the number of literals
- In algebraic expressions, each appearance of a variable is counted
 - e.g. $ab' + bc'd + a'd + e'$ contains 8 literals

Product Term

Product term is one or more literals connected by AND operators •

e.g.in $ab' + bc'd + a'd + e'$

there are 4 product terms ab' , $bc'd$, $a'd$, e'

Note: A single literal is a product term

Minterm

Minterm or **Standard product term** is a product term that includes each variable of the problem, either complemented or uncomplemented

Thus, for a function of four variables w , x , y , and z

- terms $w'xyz'$ and $wxyz$ are standard product terms
- but $wy'z$ is not (it does not contain x)

Sum Of Product SOP

SOP expression is one or more product terms connected by **OR** operators

Examples:

$w'xyz' + wx'y'z' + wx'yz + wxyz$ (4 product terms, 16 literals)

$x + w'y + wxy'z$ (3 product terms, 7 literals)

$x' + y + z$ (3 product terms, 3 literals)

wy' (1 product term, 2 literals)

z (1 product term, 1 literal)

Canonical Sum

Canonical Sum or Sum of Standard Product Terms is a SOP expression where all of terms are **standard product** terms i.e. each term contains all the variables; either complemented or uncomplemented

Example: if there are four variables, w, x, y and z, then only the first of the previous expressions is a canonical sum

Note: In order to implement algebraic expressions given in canonical sum, shall reduce them to fewest possible number of SOP terms and literals

Simplification of Canonical Sum

Consider the algebraic expression

$$x'yz' + x'yz + xy'z' + xy'z + xyz \text{ (5 terms, 15 literals)}$$

It can be simplified as

$$\begin{aligned} & x'yz' + x'yz + xy'z' + xy'z + xyz \\ &= (x'yz' + x'yz) + (xy'z' + xy'z) + xyz && \text{associative} \\ &= x'y(z' + z) + xy'(z' + z) + xyz && \text{distributive} \\ &= x'y \cdot 1 + xy' \cdot 1 + xyz && \text{complement} \\ &= x'y + xy' + xyz && \text{identity} \end{aligned}$$

More Properties of Switching Algebra

adjacency **P9a.** $ab + ab' = a$

P9b. $(a + b)(a + b') = a$

P10a. $a + a'b = a + b$

P10b. $a(a' + b) = ab$

simplification

Sum Term

Sum term is one or more literals connected by OR operators
e.g. $a + b' + c$ is a sum term

Note: A single literal e.g. b' may be considered as a sum term

Max Term

- **Maxterm** or Standard Sum Term is a sum term that includes each variable of the problem, either complemented or uncomplemented
- Thus, for a function of four variables w , x , y , and z
 - terms $w'+x'+y+z'$ and $w+x+y+z$ are standard sum terms
 - but $w'+y'+z$ is not (it does not contain x)

Product Of Sum POS

POS expression is one or more sum terms connected by AND operators

Examples:

$$(w' + x)(w + y)$$

(2 sum terms, 4 literals)

$$w.(x + y)$$

(2 sum terms, 3 literals)

$$w$$

(1 sum term, 1 literal)

$$(w + x')$$

(2 sum terms, 8 literals)

$$(w+x'+y'+z')(w'+x+y+z')$$

(1 sum term, 2 literals)

Canonical Product

Canonical Product or Product of Standard Sum Terms is a Product of sum expression where all of the terms are standard sum terms i.e. each term contains all the variables; either complemented or uncomplemented.

Example, if there are 4 variables, w, x, y and z, then only the last of the previous expressions is a canonical product

Note: In order to implement algebraic expressions given in canonical product, shall reduce them to fewest possible number of POS terms and literals

Canonical Forms

An expression may be in SOPs form, POSs form, both, or neither.
Examples are

SOP: $x'y + xy' + xyz$

POS: $(x + y')(x' + y)(x' + z')$

both: $x' + y + z$ or xyz'

neither: $x(w' + yz)$ or $z' + wx'y + v(xz + w')$

SUMMARY OF PROPERTIES OF SWITCHING ALGEBRA

P1a.	$a + b = b + a$	P1b.	$ab = ba$	Commutative
P2a.	$a + (b + c) = (a + b) + c$	P2b.	$a(bc) = (ab)c$	Associative
P3a.	$a + 0 = a$	P3b.	$a \cdot 1 = a$	Identity
P3aa.	$0 + a = a$	P3bb.	$1 \cdot a = a$	
P4a.	$a + 1 = 1$	P4b.	$a \cdot 0 = 0$	Null
P4aa.	$1 + a = 1$	P4bb.	$0 \cdot a = 0$	
P5a.	$a + a' = 1$	P5b.	$a \cdot a' = 0$	Complement
P5aa.	$a' + a = 1$	P5bb.	$a' \cdot a = 0$	
P6a.	$a + a = a$	P6b.	$a \cdot a = a$	Idempotency
P7.	$(a')' = a$			Involution
P8a.	$a(b + c) = ab + ac$	P8b.	$a + bc = (a + b)(a + c)$	Distributive
P9a.	$ab + ab' = a$	P9b.	$(a + b)(a + b') = a$	Adjacency
P9aa.	$a'b' + a'b + ab + ab' = 1$	P9bb.	$(a' + b')(a' + b)(a + b)(a + b') = 0$	
P10a.	$a + a'b = a + b$	P10b.	$a(a' + b) = ab$	Simplification
P11a.	$(a + b)' = a'b'$	P11b.	$(ab)' = a' + b'$	DeMorgan
P11aa.	$(a + b + c \dots)' = a'b'c' \dots$	P11bb.	$(abc \dots)' = a' + b' + c' \dots$	
P12a.	$a + ab = a$	P12b.	$a(a + b) = a$	Absorption
P13a.	$at_1 + a't_2 + t_1t_2 = at_1 + a't_2$	P13b.	$(a + t_1)(a' + t_2)(t_1 + t_2) = (a + t_1)(a' + t_2)$	Consensus
P14a.	$ab + a'c = (a + c)(a' + b)$			

Simplification of Canonical Product

Example: Simplification of $(w' + x' + y + z')(w' + x' + y + z)(w + x' + y + z')$

$$\begin{aligned} & (w' + x' + y + z')(w' + x' + y + z)(w + x' + y + z') \\ &= ((w' + x' + y) + z'.z) (w + x' + y + z') && \text{Using P6b} \\ &= ((w' + x' + y) + 0) (w + x' + y + z') && \text{Using P5b} \\ &= (w' + x' + y)(w + x' + y + z') && \text{Using P3a} \\ &= (x' + y + w')(x' + y + w + z') && \text{Using P2a} \\ &= x' + y + w'.(w + z') && \text{Using P8b} \\ &= x' + y + (w'.w + w'.z') && \text{Using P8a} \\ &= x' + y + (0 + w'.z') && \text{Using P5b} \\ &= x' + y + (w'.z') && \text{Using P3a} \\ &= (x' + y + w')(x' + y + z') && \text{Using P8b} \end{aligned}$$

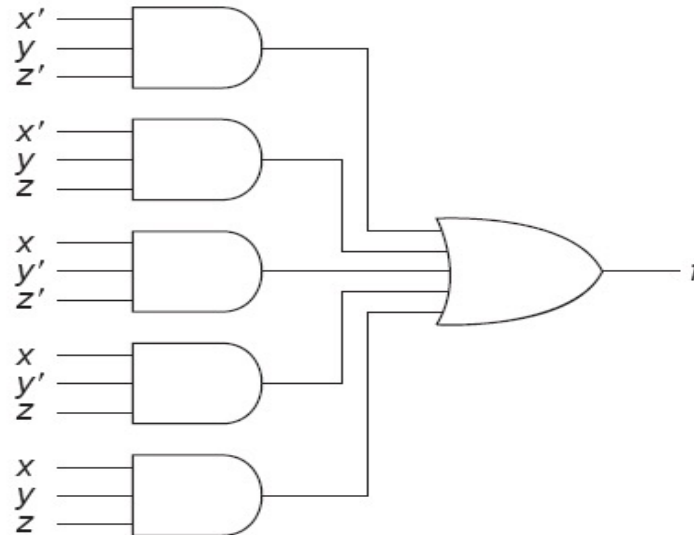
Implementing Switching Functions

First, implement switching functions using **AND**, **OR** and **NOT** gates. Consider the following function:

$$f = x'yz' + x'yz + xy'z' + xy'z + xyz$$

A block diagram of the implementation of this function is

Figure 2.7 Block diagram of f in sum of standard products form.



Implementing Switching Functions

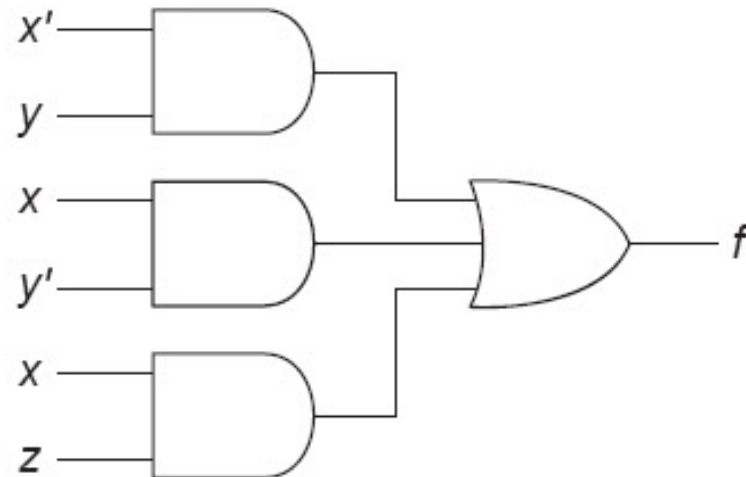
- Each of the product terms is formed by an AND gate
 - In this example, all of the AND gates have 3 inputs
 - Outputs of the AND gates are used as inputs to an OR gate
- This is an example of a two-level (2L) circuit
 - Number of levels is the maximum number of gates through which a signal must pass from the input to the output
- All signals go first through an AND gate and then through an OR gate
- When inputs are available both uncomplemented and complemented, implementations of both SOP and POS expressions result in 2L circuits

Implementing Simplified Switching Functions

We have already seen that one of the simplified versions of previous function with minimum SOP expression is $f = x'y + xy' + xz$

Implementation of this simplified function results in less complex circuit which is as follows:

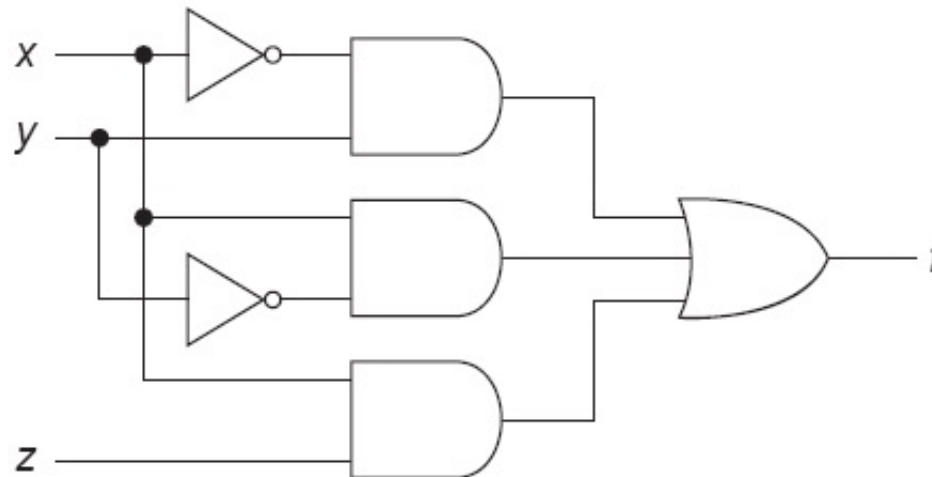
Figure 2.8 Minimum sum of product implementation of f .



Implementing Simplified Switching Functions

- If complemented inputs are not available, then an inverter (a NOT gate) is needed for each input that is required to be complemented
- Circuit of Figure 2.9 shows NOT gates that must be added to circuit of Figure 2.8

Figure 2.9 Circuit with only uncomplemented inputs.

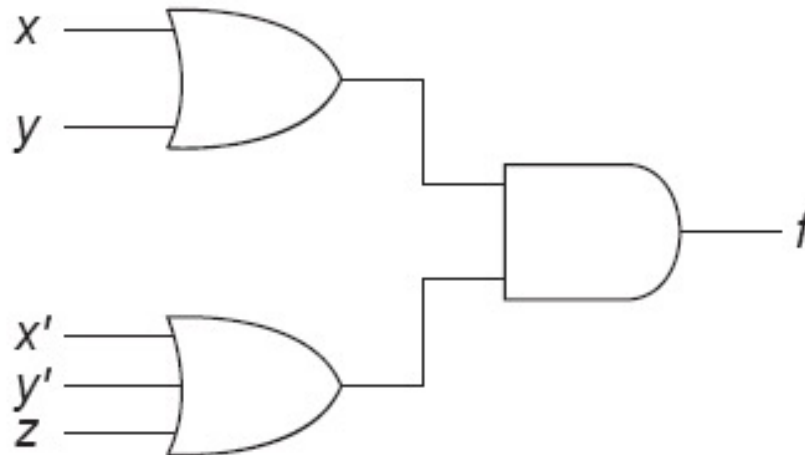


Implementing Simplified Switching Functions

POS expression corresponds to a two-level OR–AND network

- e.g., the POS expression $f = (x + y)(x' + y' + z)$ is implemented as follows:

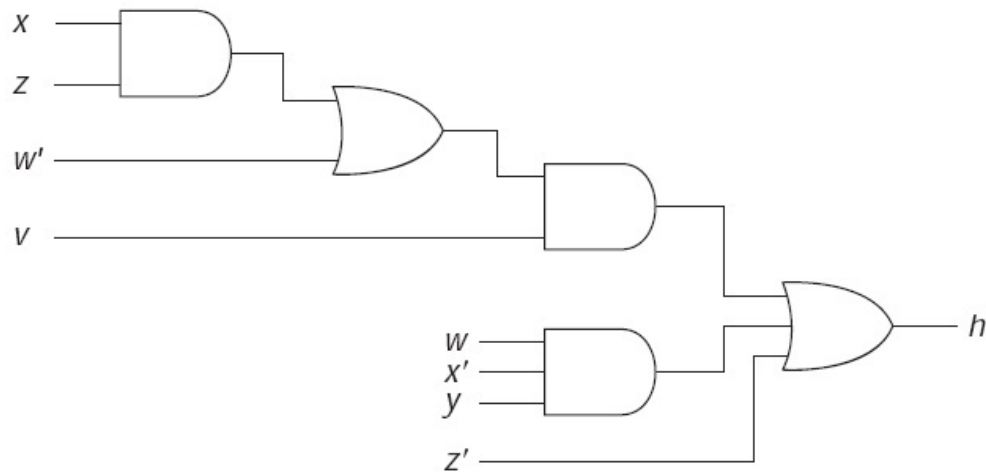
Figure 2.10 A product of sums implementation.



Implementing Functions that are neither in SOP nor in POS

- When we implement functions that are neither in SOP nor in POS form, the resulting circuits are more than two levels.
- e.g., the function: $h = z' + wx'y + v(xz + w')$
- This function is implemented (with 4-level circuit) as follows:

Figure 2.11 A multilevel circuit.

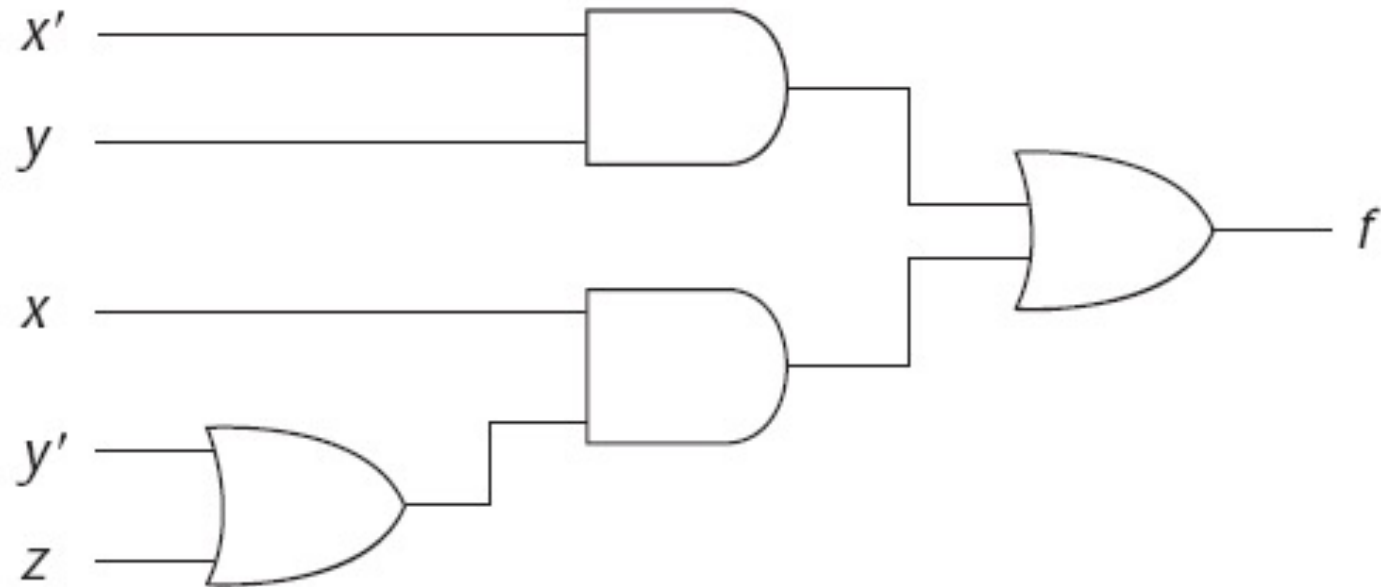


Implementing Functions that are neither in SOP nor in POS

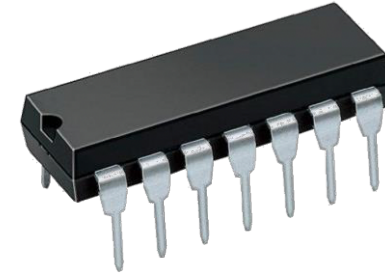
Consider another function that is neither in SOP nor in POS

$$f = x'y + x(y' + z)$$

This function is implemented in 3-level circuit as follows:



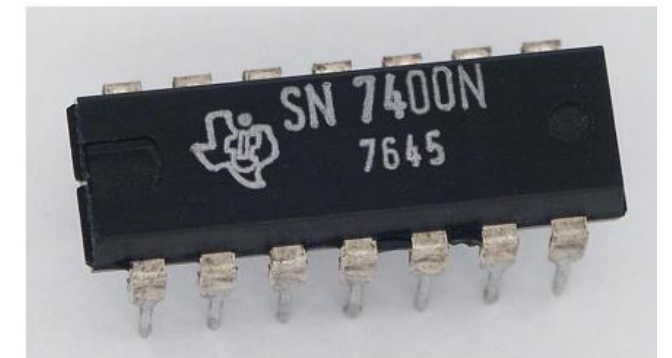
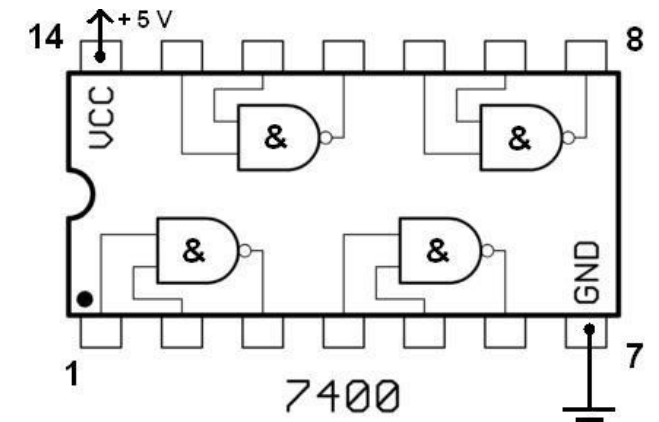
Integrated Circuits



- Gates are typically available in dual in-line pin packages (DIPs) of 14 connector pins; these packages are often referred to as **chips**
 - Larger packages of 16, 18, 22, and more pins are used for more complex logic
- These packages contain **Integrated Circuits (ICs)** that are categorized as
 - Small-scale integration (**SSI**); contains just a few gates
 - Medium Scale Integrated (**MSI**); contains as many as 100 gates
 - Large-Scale Integration (**LSI**), Very Large-Scale Integration (**VLSI**), and Giga- Scale Integration (**GSI**) are used for even more complex packages, including complete computers

Integrated Circuits

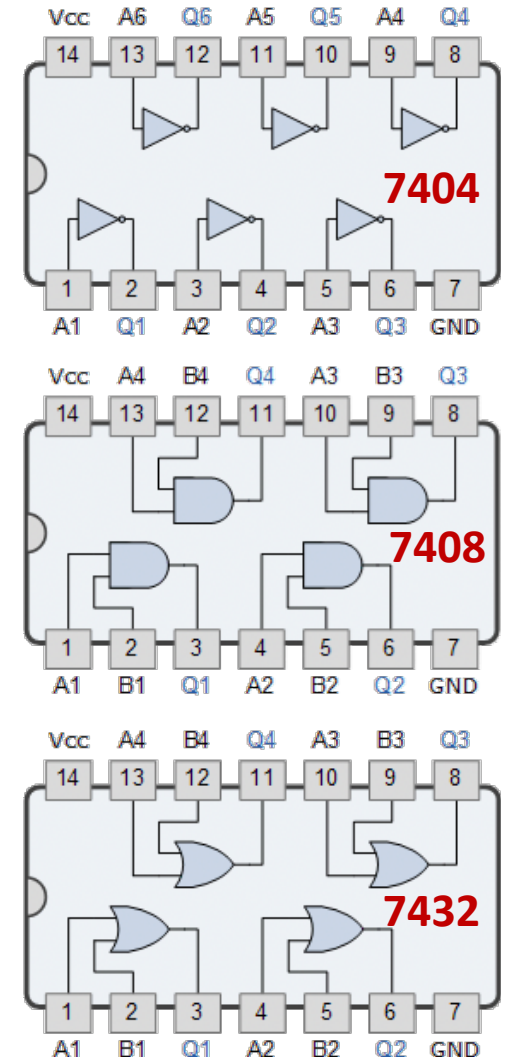
- Two of connector pins used to provide power to chip
 - Leaves 12 pins for logic connections (on a 14-pin chip)
- Thus, we can fit 4 two-input gates on a chip
 - Each gate has 2 input connections and 1 output connection
- Similarly, there are enough pins for
 - 6 one-input gates (NOTs)
 - 3 three-input gates
 - 2 four-input gates (with two pins unused)



Common Integrated Circuits

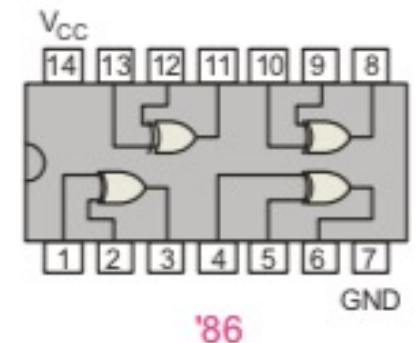
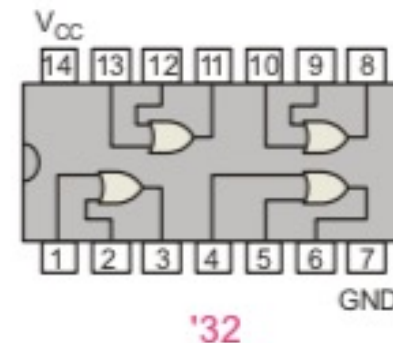
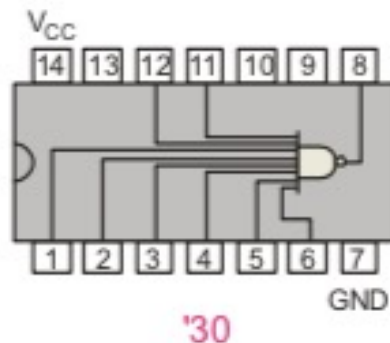
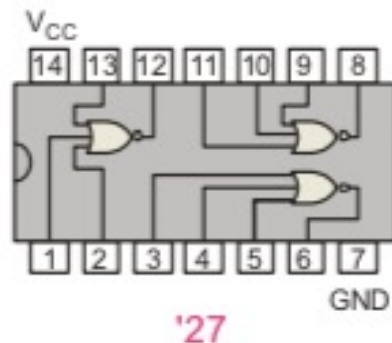
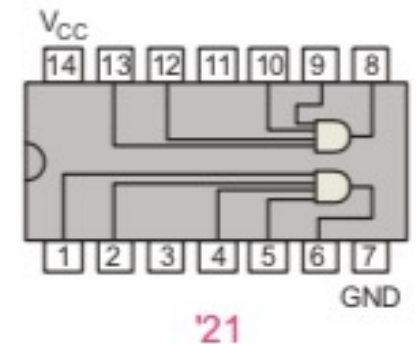
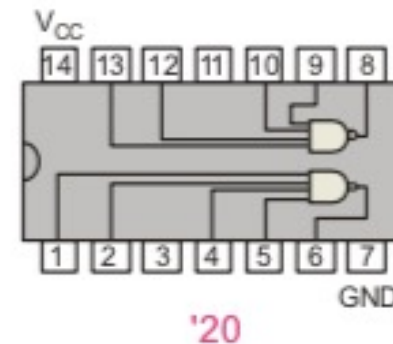
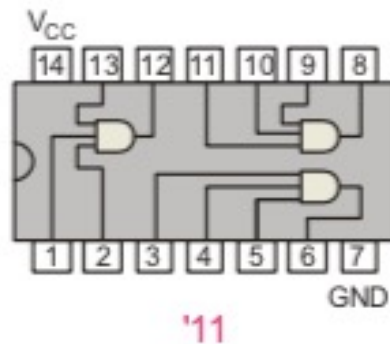
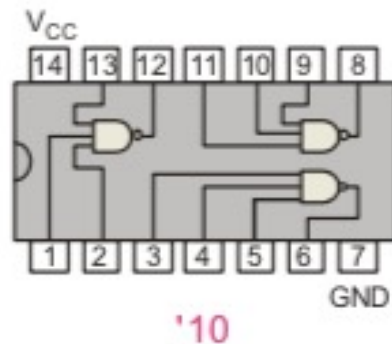
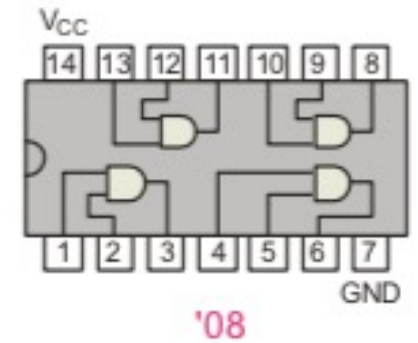
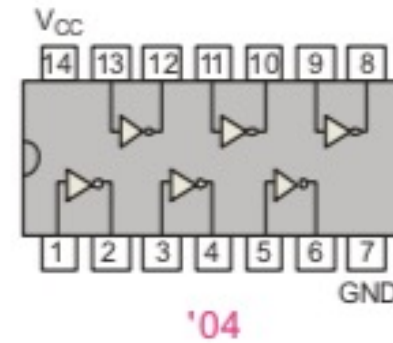
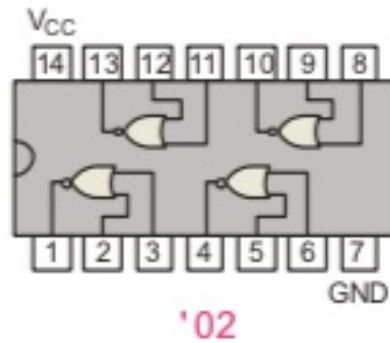
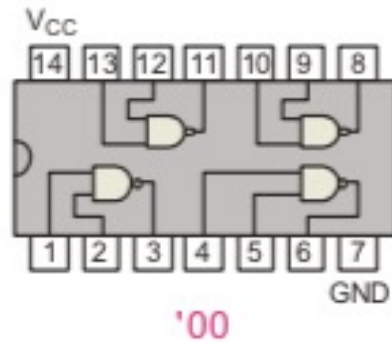
A list of the common AND, OR, and NOT integrated circuits that might be encountered in laboratory is

- **7404** 6 (hex) NOT gates
- **7408** 4 (quadruple) two-input AND gates
- **7411** 3 (triple) three-input AND gates
- **7421** 2 four-input (dual) AND gates
- **7432** 4 (quadruple) two-input OR gates



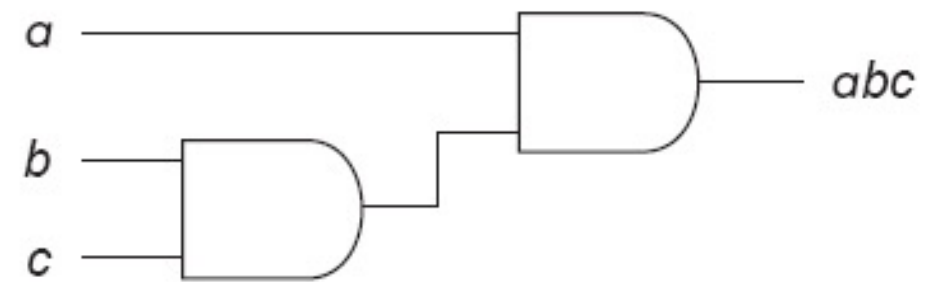
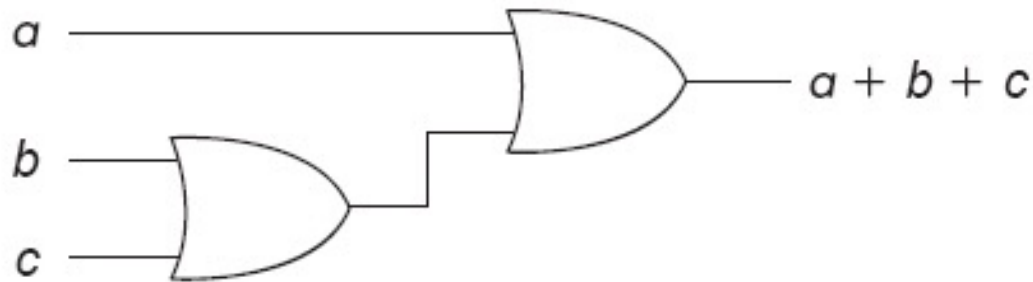
Fixed Function Logic

Some
common gate
configurations



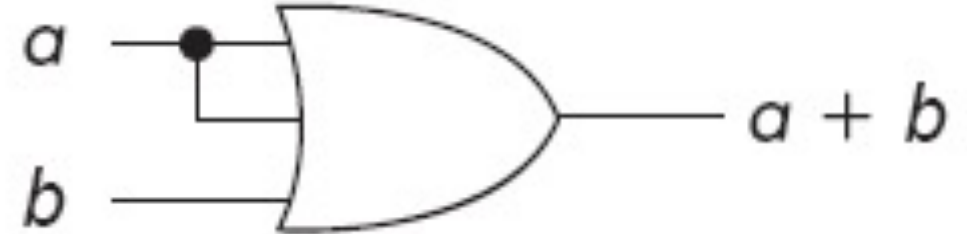


If a three-input OR (or AND) is needed, and only two-input ones are available, it can be constructed as follows:



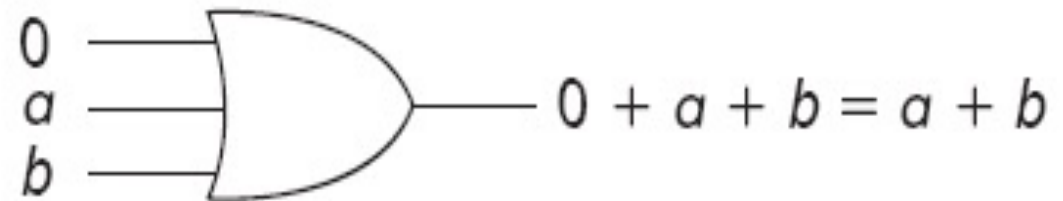
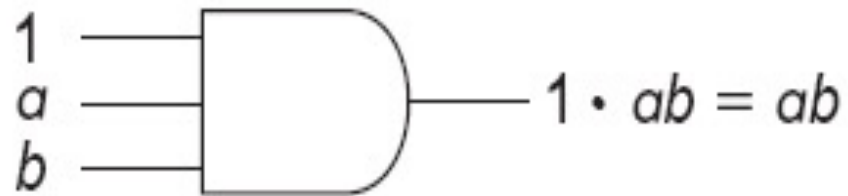


Also, if we need a two-input gate and there is a leftover three-input one (because they come three to a package), we can connect the same signal to two of the inputs (since $a.a = a$, and $a + a = a$)





Also, we could connect a logic 1 (5 V) to one of the inputs of an AND gate or a logic 0 (ground) to one of the inputs of an OR gate, as:



Positive Logic and Negative Logic

- Logic-0 and logic-1 are represented by two voltages: often 0 (ground) and 5 V
- Commonly, higher voltage is used to represent 1 and lower voltage to represent 0; referred to as *positive logic*
- Opposite choice is also possible, that is, use higher voltage to represent 0 and lower voltage to represent 1; referred to as *negative logic*
- Consider TT of Table 2.9a, where the behavior of the gate is described just in terms of high (H) and low (L)

Positive Logic and Negative Logic

Table 2.9

a. High/Low

<i>a</i>	<i>b</i>	<i>f</i>
L	L	L
L	H	H
H	L	H
H	H	H

b. Positive logic

<i>a</i>	<i>b</i>	<i>f</i>
0	0	0
0	1	1
1	0	1
1	1	1

c. Negative logic

<i>a</i>	<i>b</i>	<i>f</i>
1	1	1
1	0	0
0	1	0
0	0	0

- Positive logic interpretation of Table 2.9b produces TT for an OR gate
- Negative logic interpretation of Table 2.9c is that of an AND gate
- **Hence**, from the above tables, we can conclude that Positive Logic OR gate is equal to Negative Logic AND gate

DeMorga's Law

DeMorgan's Laws state that

$$\text{P11a. } (a + b)' = a'b'$$

$$\text{P11b. } (ab)' = a' + b'$$

We can prove these laws with the help of truth tables as

Table 2.10 Proof of DeMorgan's theorem.

a	b	$a + b$	$(a + b)'$	a'	b'	$a'b'$	ab	$(ab)'$	$a' + b'$
0	0	0	1	1	1	1	0	1	1
0	1	1	0	1	0	0	0	1	1
1	0	1	0	0	1	0	0	1	1
1	1	1	0	0	0	0	1	0	0
			11a			11a		11b	11b

Generalization

In general, DeMorgan's laws can be extended to more than one variables to get

$$\text{P11aa. } (a + b + c \dots)' = a'b'c' \dots$$

$$\text{P11bb. } (abc \dots)' = a' + b' + c' \dots$$

Complement of an Expression

Can be obtained by repeatedly applying DeMorgan's laws or following the rules:

1. Complement each variable (that is, a to a' or a' to a)
2. Replace 0 by 1 and 1 by 0.
3. Replace AND by OR and OR by AND, being sure to preserve the order of operations; it sometimes requires additional parentheses

Note: If the function is in SOP form, its complement will be in POS form and the complement of a POS expression will be an SOP one

Example

Consider the following function

$$f = ab'(c + d'e) + a'bc'$$

Its complement can be obtained as follows:

$$\begin{aligned} f' &= [ab'(c + d'e) + a'bc']' \\ &= [ab'(c + d'e)]' [a'bc']' \\ &= [a' + b + (c + d'e)'] [a + b' + c] \\ &= [a' + b + c'(d'e)'] [a + b' + c] \\ &= [a' + b + c'(d + e')] [a + b' + c] \end{aligned}$$

Converting TT into Algebraic Expression

Sometimes, we need to convert truth tables into algebraic expressions and vice versa

Consider the two-variable truth table of Table 2.11

Table 2.11 A two-variable truth table.

<i>a</i>	<i>b</i>	<i>f</i>
0	0	0
0	1	1
1	0	1
1	1	1

cont.

- From the table, we see that f is 1
if $a = 0$ AND $b = 1$ OR
if $a = 1$ AND $b = 0$ OR
if $a = 1$ AND $b = 1$
- However, this is the same as saying f is 1
if $a' = 1$ AND $b = 1$ OR
if $a = 1$ AND $b' = 1$ OR
if $a = 1$ AND $b = 1$

• But $a' = 1$ AND $b = 1$ is the same as saying $a'b = 1$
and thus f is 1 if $a'b = 1$ OR if $ab' = 1$ OR if $ab = 1$

- That finally produces the expression:

$$f = a'b + ab' + ab$$

Table 2.11 A two-variable truth table.

a	b	f
0	0	0
0	1	1
1	0	1
1	1	1

Explanation

- Each row of TT corresponds to a product term
- An SOP expression is formed by ORing those product terms that correspond to the rows of the truth table for which the function is 1
- Each product term has each variable included, with that variable complemented when the entry in the input column for that variable contains a 0 and uncomplemented when it contains a 1
- Thus, for example, row 10 produces the term ab'
- These product terms include all variables, so, they are called **minterms**

Explanation cont.

- Minterms are often referred to by number, by just converting the binary number in the input row of the truth table to decimal
- So, both of the following notations are common

$$f(a, b) = m_1 + m_2 + m_3$$

$$f(a, b) = \Sigma m(1, 2, 3)$$

Explanation cont.

- Table shows **minterms** and **minterm** numbers that are used for all functions of three variables: A, B and C

- For a specific function

- those terms for which the function is 1 are used to form SOP expression for **f**

- and those terms for which the function is 0 are used to form an SOP expression for **f'**

- Can complement **f'** to form a POS expression for **f**

Table 2.12 Minterms.

<i>ABC</i>	Minterm	Number
0 0 0	$A'B'C'$	0
0 0 1	$A'B'C$	1
0 1 0	$A'BC'$	2
0 1 1	$A'BC$	3
1 0 0	$AB'C'$	4
1 0 1	$AB'C$	5
1 1 0	ABC'	6
1 1 1	ABC	7

Example

Consider TT showing both the function f and its complement f'

We can write $f(A, B, C)$

$$f(A, B, C) = \sum m(1, 2, 3, 4, 5)$$

$$= A'B'C + A'BC' + A'BC + AB'C' + AB'C$$

Now, writing minterms against the 1's of f' , we get

$$f'(A, B, C) = \sum m(0, 6, 7)$$

$$= A'B'C' + ABC' + ABC$$

We can complement f' to get a product of maxterms as

$$f = (f')' = (A+B+C)(A'+B'+C)(A'B'C')$$

ABC	f	f'
0 0 0	0	1
0 0 1	1	0
0 1 0	1	0
0 1 1	1	0
1 0 0	1	0
1 0 1	1	0
1 1 0	0	1
1 1 1	0	1

Another Example

- From implementation point of view, the sum of minterms and product of maxterms expressions are not suitable.
- So, we need to reduce the functions into minimum terms and literals.
- Thus, we can reduce the above function, given in sum of minterms, into minimum numbers of terms and literals as:

$$\begin{aligned}f &= A'B'C + A'BC' + A'BC + AB'C' + AB'C \\&= A'B'C + A'B + AB' \\&= A'C + A'B + AB' \\&= B'C + A'B + AB'\end{aligned}$$

[P9a, P9a]

Example

Similarly, we can reduce the second function, given in product of maxterms, into minimum number of terms and literals as:

$$\begin{aligned} f &= (A + B + C)(A' + B' + C)(A' + B' + C') \\ &= (A + B + C)(A' + B' + C.C') \\ &= (A + B + C)(A' + B') \text{ (Since, } C.C' = 0) \end{aligned}$$

Exercises

Construct truth tables and then write the algebraic expressions for the following systems:

1. A system with three inputs A, B, and C, and one output, Z, such that $Z = 1$ iff two of the inputs are 1.
2. A system to do 1 bit of binary addition. It has three inputs (2 bits to be added plus carry from next lower order bit) and produces two outputs: a sum bit and a carry to the next higher order position.

General Approach to Switching Functions

For n variables, we can have 2^{2^n} different switching functions.

- So, for two variables, there are 16 possible truth tables, resulting in 16 different functions.
- The truth table of Table 2.13 shows all of these functions

General Approach to Switching Functions

Set of functions, f_0 to f_{15} , reduced to minimum SOP form are as follows:

Table 2.13 All two-variable functions.

a	b	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

$$f_0 = 0$$

$$f_1 = ab$$

$$f_2 = ab'$$

$$f_3 = a$$

$$f_4 = a'b$$

$$f_5 = b$$

$$f_6 = a'b + ab'$$

$$f_7 = a + b$$

$$f_8 = a'b'$$

$$f_9 = a'b' + ab$$

$$f_{10} = b'$$

$$f_{11} = a + b'$$

$$f_{12} = a'$$

$$f_{13} = a' + b$$

$$f_{14} = a' + b'$$

$$f_{15} = 1$$

NAND Gate

Truth table for **NAND** (Not AND) gate is as follows:

a	b	$(a.b)'$
0	0	1
0	1	1
1	0	1
1	1	0

Symbols for NAND gate are:

Figure 2.12 NAND gates.

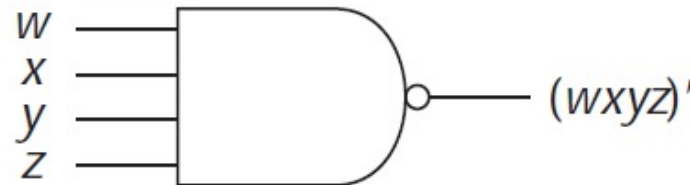
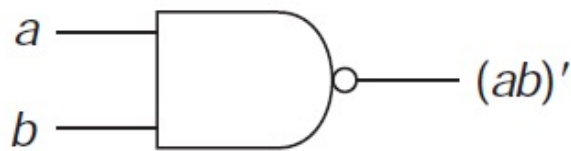
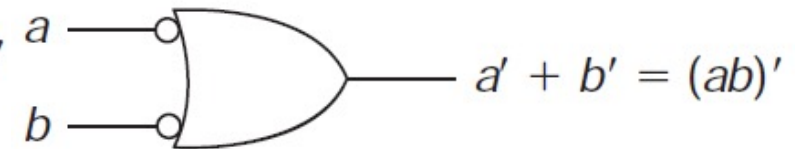


Figure 2.13 Alternative symbol for NAND.



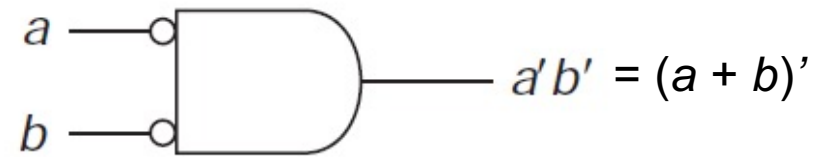
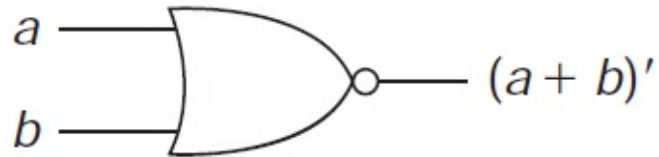
NOR Gate

Truth table for **NOR** (Not OR) gate is as follows:

a	b	$(a+b)'$
0	0	1
0	1	0
1	0	0
1	1	0

Symbols for NOR gate are:

Figure 2.14 Symbols for NOR gate.



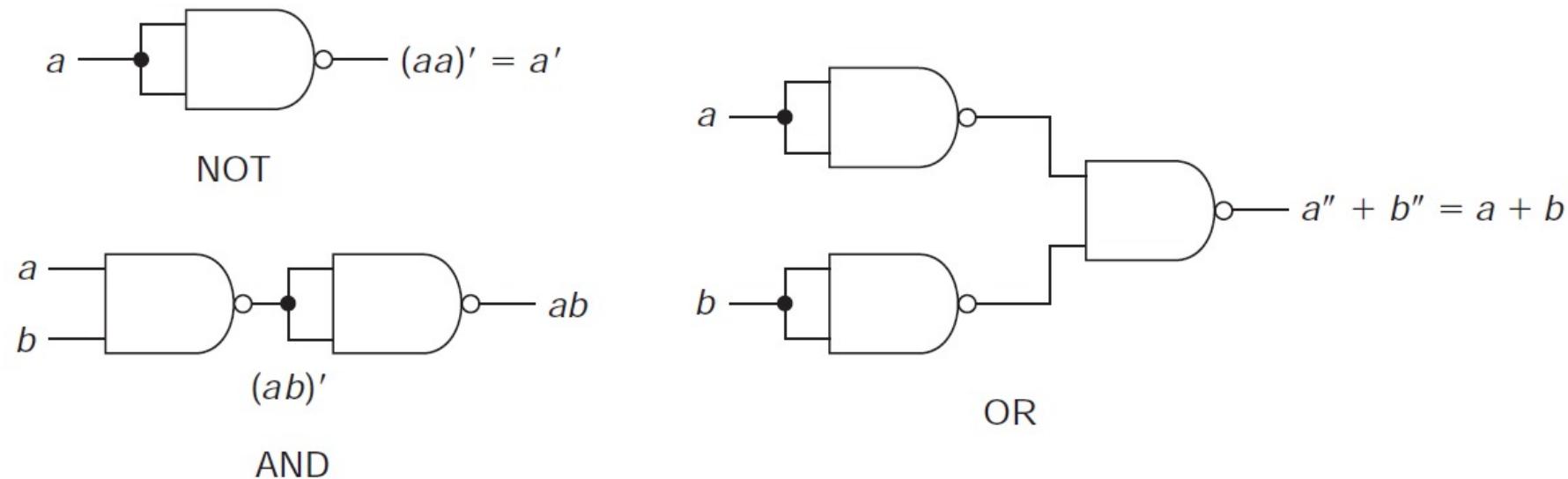
Why use NAND and NOR Gates?

- Many electronic implementations naturally invert (complement) signals
 - **NAND** is more convenient to implement than **AND**
- Most important reason is that with either **NAND** or **NOR**, only one type of gate is required
- On the other hand, both **AND** and **OR** gates are required; and, often, **NOT** gates are needed, as well

Using NAND Gate

The following circuits represent the behavior of NOT, AND and OR gates, using only NAND gates

Figure 2.15 Functional completeness of NAND.

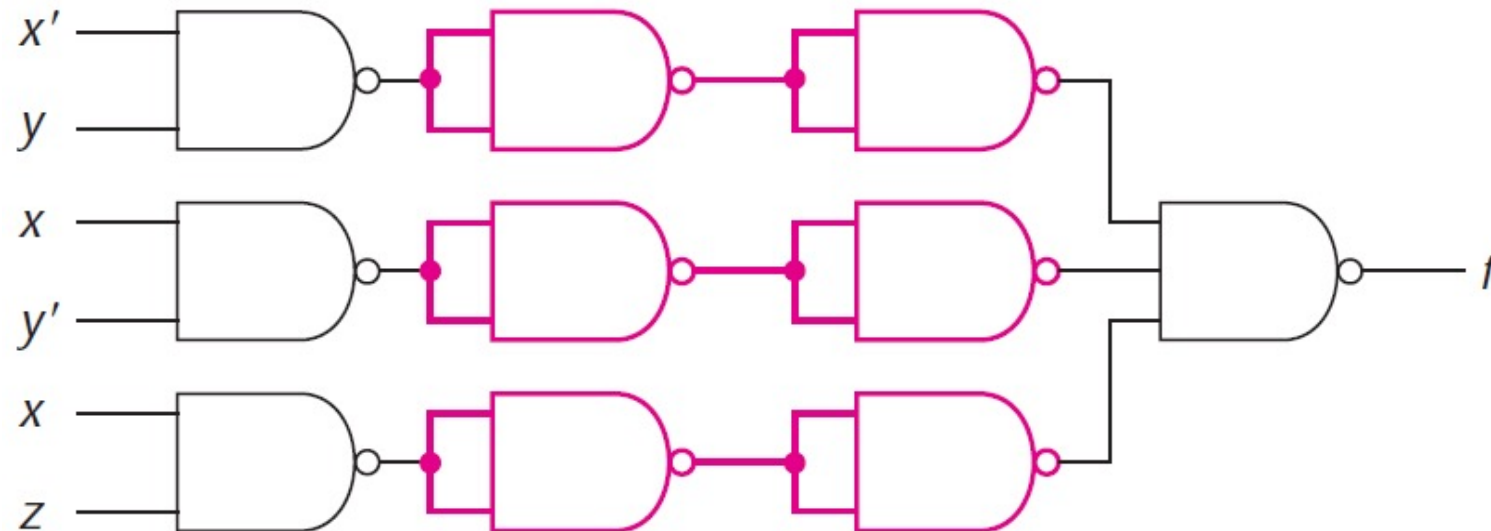


NAND Gate Implementation

Consider the expression $f = x'y + xy' + xz$

It can be implemented using only **NAND** gates as follows:

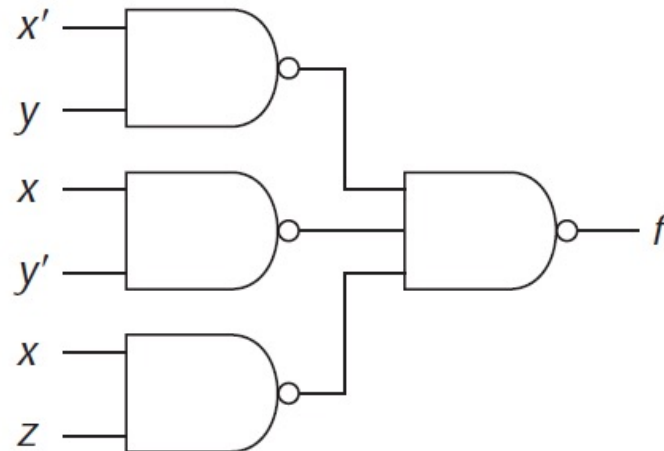
Figure 2.16 NAND gate implementation.



NAND Gate Implementation

- The colorful **NAND** gates in a row represent two **NOT** gates; one after the other, so, they cancel each other
- Hence, they can be eliminated yielding the following simplified circuit

Figure 2.17 Better NAND gate implementation.



NAND Gate Implementation

- Process is simplified when circuit consists of **AND** & **OR** gates such that
 1. **Output** of the circuit comes from an **OR**
 2. **Inputs** to all **OR** gates come either from a system input or from the output of an **AND**
 3. **Inputs** to all **AND** gates come either from a system input or from the output of an **OR**
- All gates are replaced by **NAND** gates
- Any input coming directly into an **OR** is complemented

NAND Gate Implementation

To implement a circuit using only **NAND** gates, function should be in **SOP** form

1. Pass every **AND** term, through a separate **NAND** gate
2. Add one **NAND** gate at the end and pass all the outputs of previous **NAND** gates, as an input to this **NAND** gate
3. Any input coming directly into an **OR** is complemented

Converting NAND Implementation Into AND-OR Implementation

- If we use alternate symbol for last NAND gate, we get following figure:

Figure 2.17 Better NAND gate implementation.

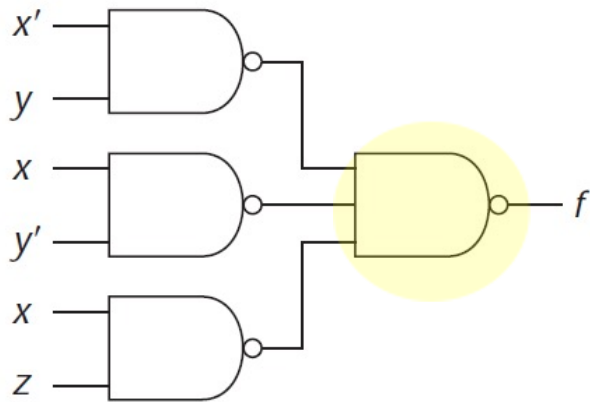


Figure 2.13 Alternative symbol for NAND.

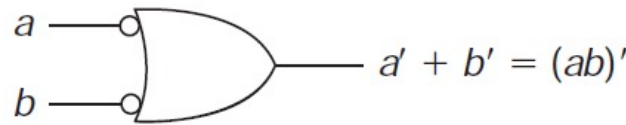
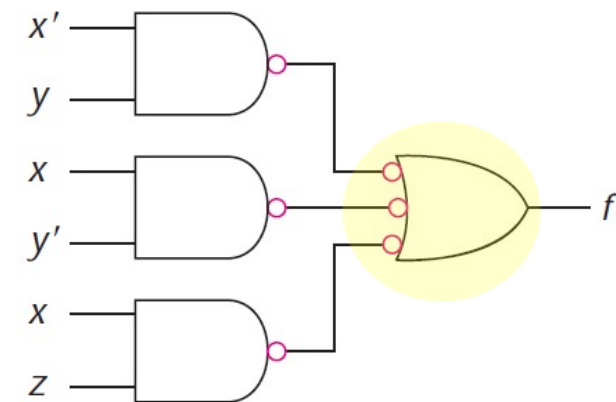


Figure 2.18 Double NOT gate approach.



- Bubbles connecting the same line cancel each other
- As a result, we get the AND-OR implementation of function f. HOW?

Multilevel NAND Implementation

the circuit for $h = z' + wx'y + v(xz + w')$

can be implemented using only NAND gates as follows:

Figure 2.11 A multilevel circuit.

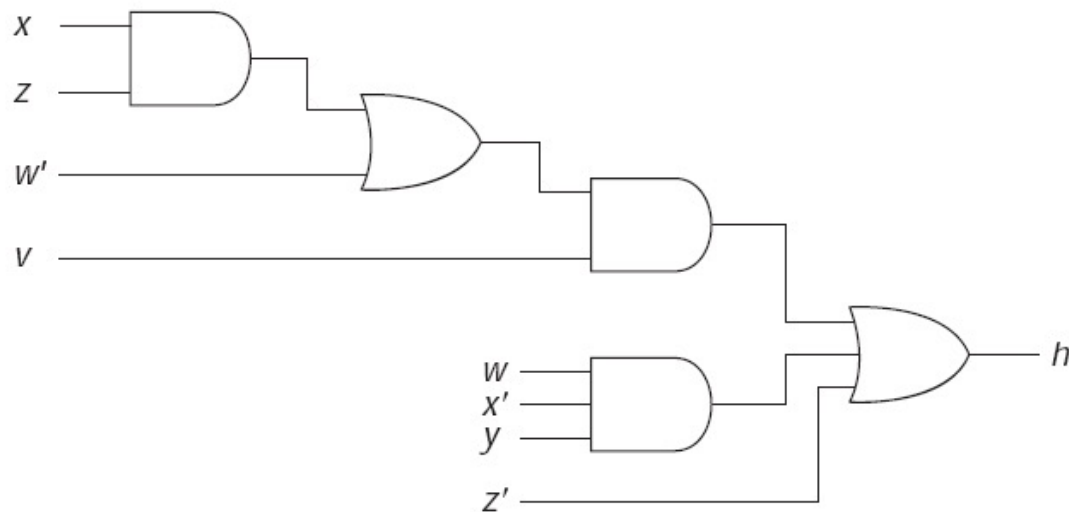
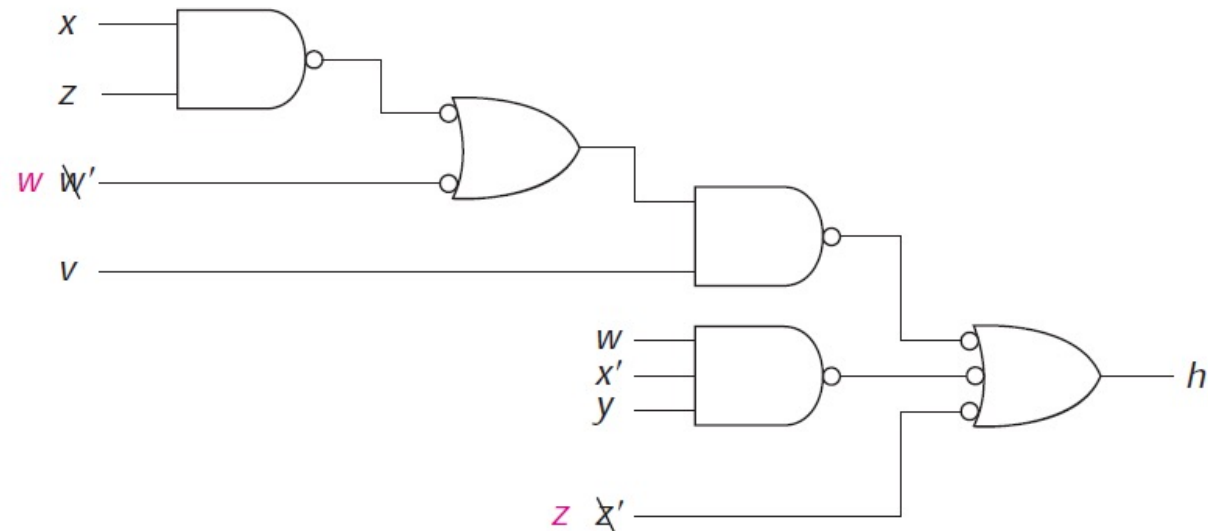


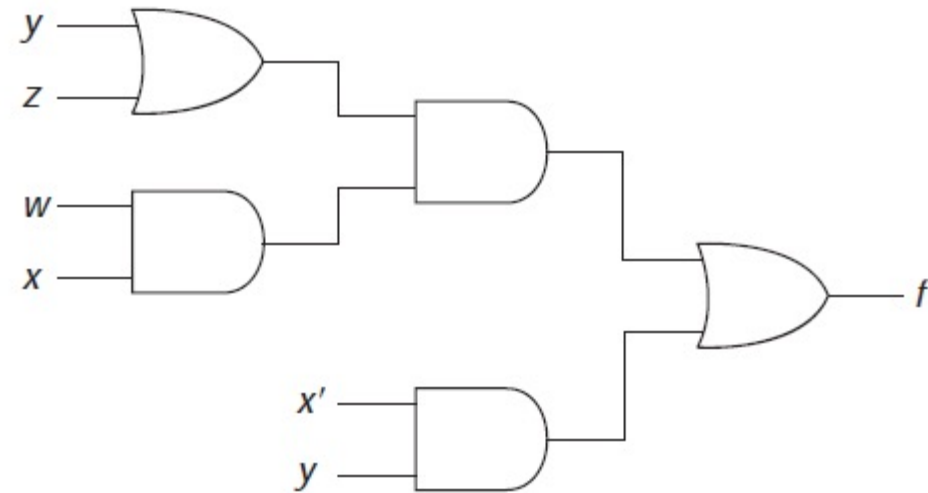
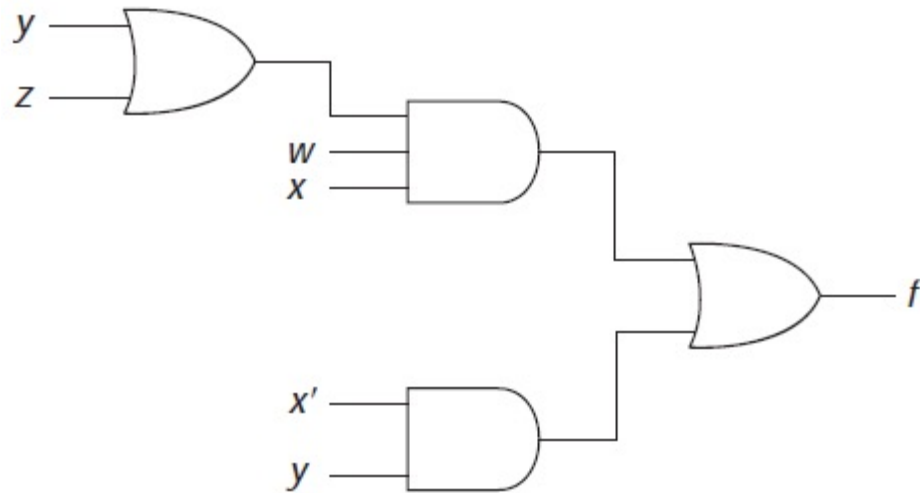
Figure 2.19 A multilevel NAND implementation.



Example

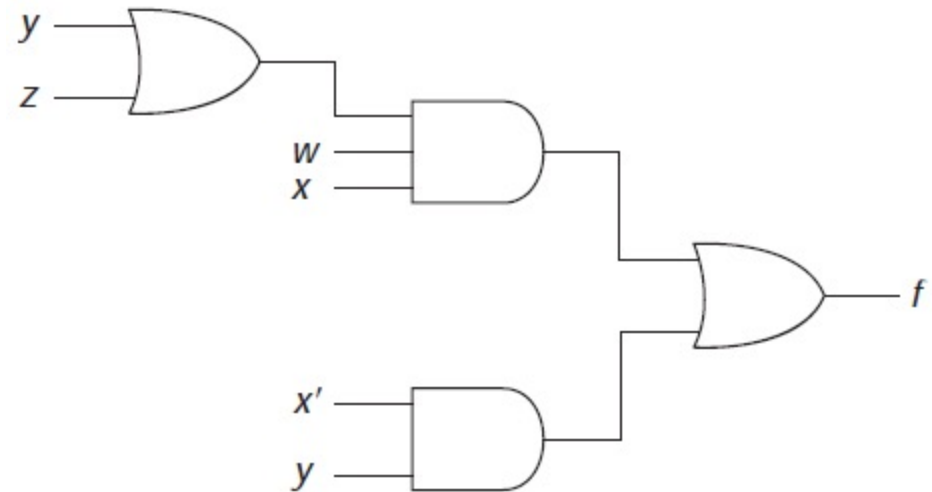
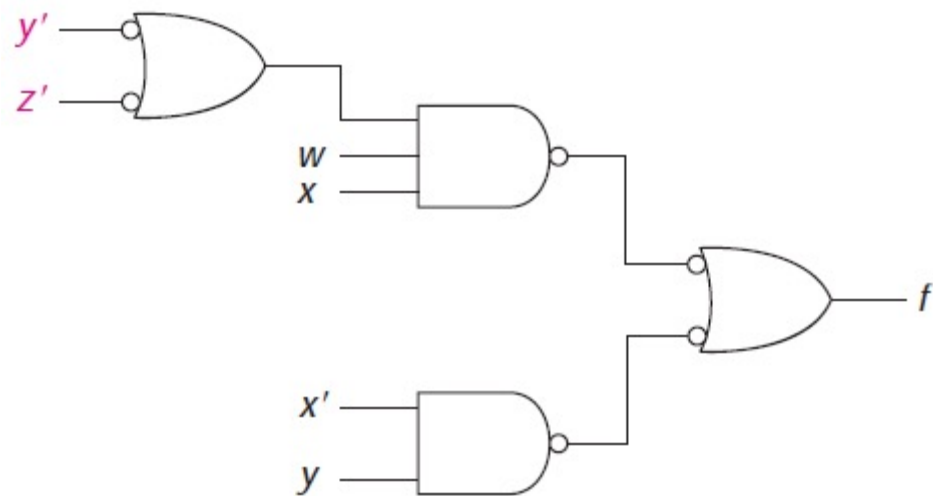
Consider the following function: $f = wx(y + z) + x'y$

It can be implemented using AND-OR gates in the following two ways:



Example cont.

First version can be directly converted to NAND gates as:

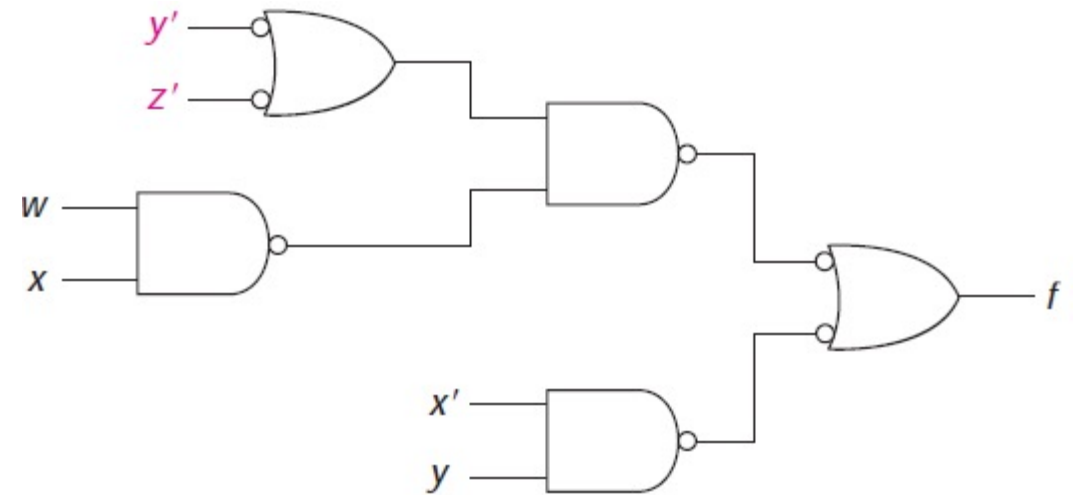
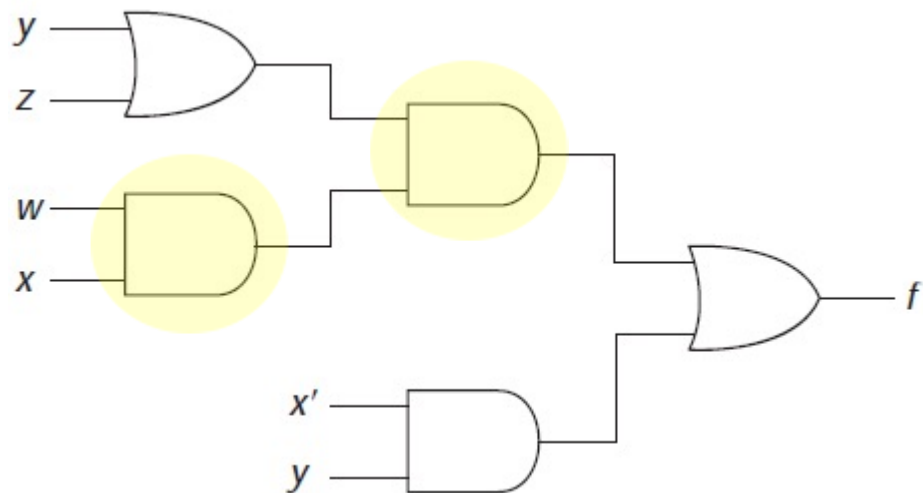


$$f = wx(y + z) + x'y$$

Example cont.

Second version **cannot** be converted to **NAND** gates without adding an extra **NOT** gate

- Violates rule: AND gets an input from another AND
- Thus, this circuit would get the following form:



Try to write output of each gate, in order to derive the function f .

NOR Implementation

In order to implement a circuit using only **NOR** gate, the function should be **POS** form

When this function is directly implemented using **AND-OR** gates, we use **OR** gates at the first level and one **AND** gate at the second level

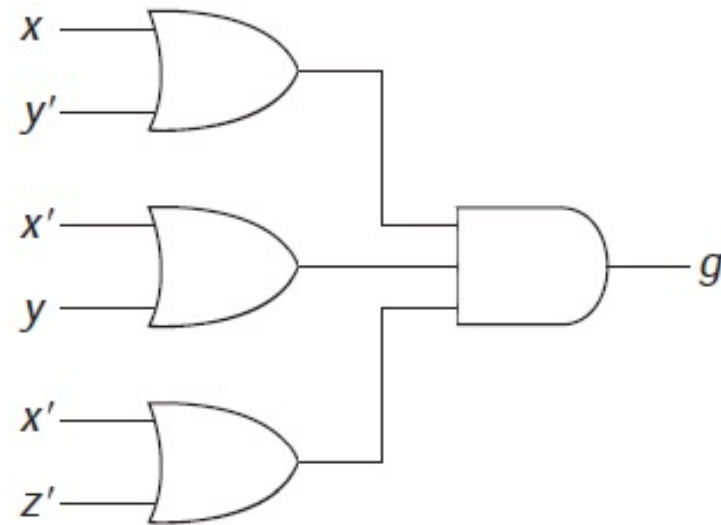
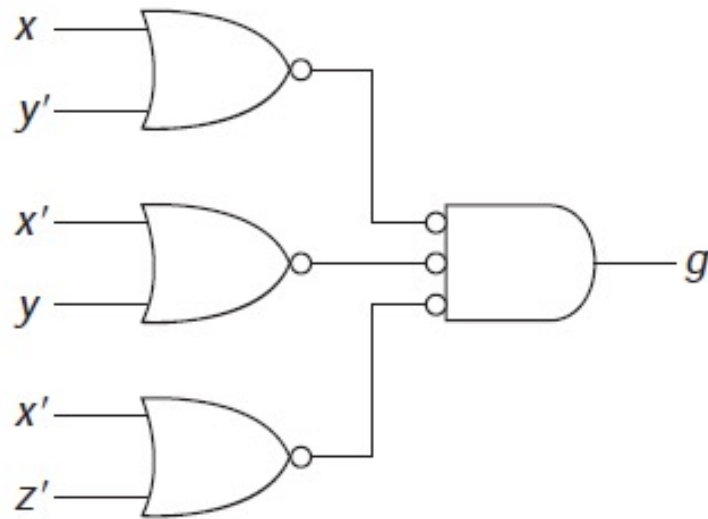
NOR Implementation

- When we have a circuit consisting of **AND** and **OR** gates such that
 1. Output of the circuit comes from an **AND**
 2. Inputs to **OR** gates come either from system input or from output of **AND**
 3. Inputs to **AND** gates come either from system input or from output of **OR**
- All gates can be converted to **NOR** gates
- If an input comes directly into an **AND** gate, it is complemented

Example

Consider the following function: $g = (x + y')(x' + y)(x' + z')$

It can be implemented using only NOR gates as



Bubbles connecting same line cancel each other, as a result we can get the simple AND-OR logic diagram

Truth Tables of XOR and XNOR Gates

Exclusive-OR gives 1 when either of the inputs is 1 and gives 0 if both inputs are 0 or both are 1.

— Its expression is: $a \oplus b = a'b + ab'$

a	b	$(a \oplus b)$
0	0	0
0	1	1
1	0	1
1	1	0

Exclusive-NOR gives 1 when both inputs are 0 or both inputs are 1.

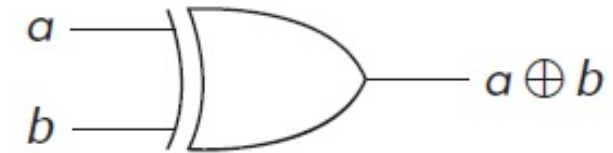
— Its expression is: $(a \oplus b)' = a'b' + ab$

a	b	$(a \oplus b)'$
0	0	1
0	1	0
1	0	0
1	1	1

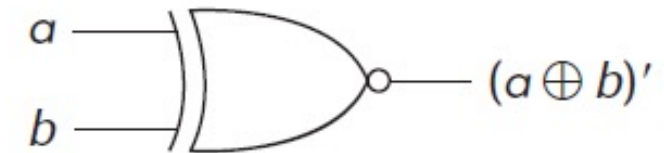
Symbols of XOR and XNOR

- Symbols for **Exclusive-OR** and **Exclusive-NOR**
- Exclusive-NOR gate is sometimes referred to as **Comparator**

Figure 2.20 (a) An Exclusive-OR gate. (b) An Exclusive-NOR gate.



(a)



(b)

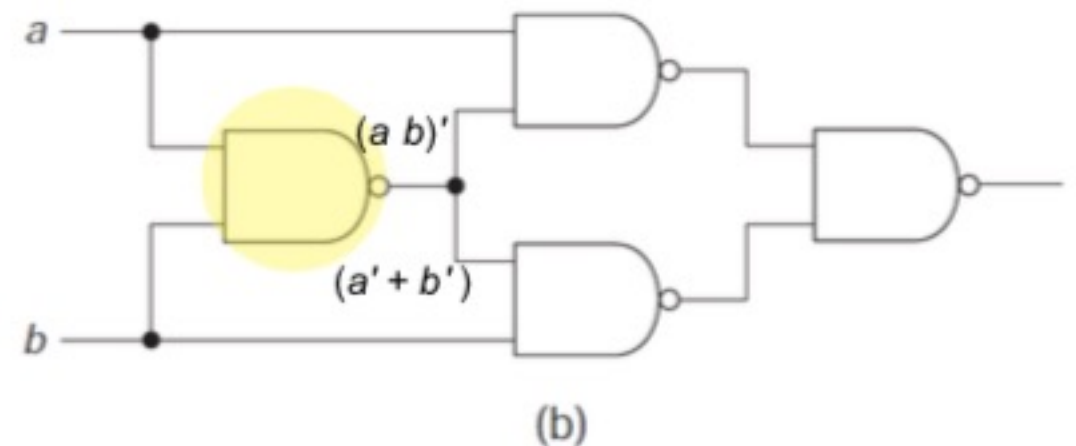
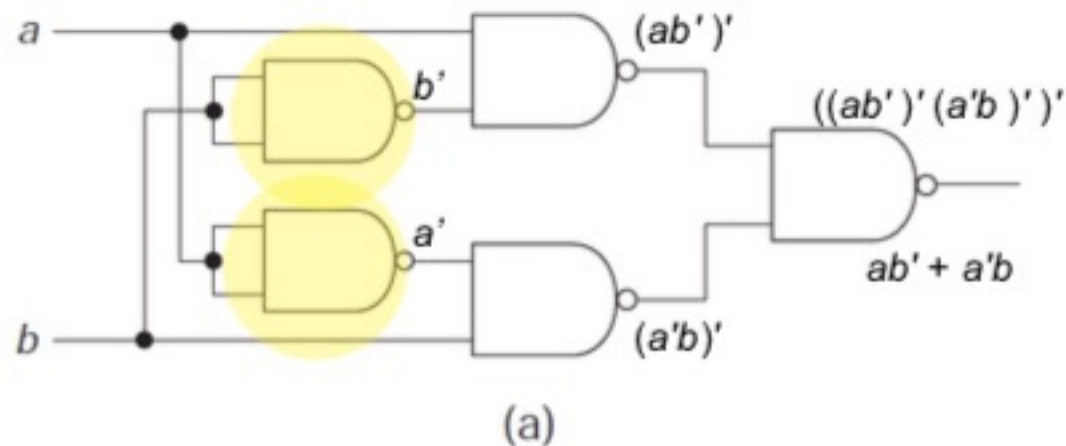
NAND Gate Implementation of Exclusive-OR

NAND gate implementation of XOR is shown in Figure 2.21a

- Only uncomplemented inputs are assumed
- Two NOT gates (NANDs) replaced by single gate, as shown in Figure 2.21b, since

$$a(a' + b') + b(a' + b') = aa' + ab' + ba' + bb' = ab' + a'b$$

Figure 2.21 Exclusive-OR gates.



$$a \oplus b = ab' + a'b$$

Properties of Exclusive-OR

Some useful properties of the Exclusive-OR are

$$(a \oplus b)' = (a'b + ab')' = (a + b')(a' + b) = a'b' + ab$$

$$a' \oplus b = (a')'b + (a')b' = ab + a'b' = (a \oplus b)'$$

$$(a \oplus b') = (a \oplus b)'$$

$$a \oplus 0 = a = (a' \cdot 0 + a \cdot 1)$$

$$a \oplus 1 = a' = (a' \cdot 1 + a \cdot 0)$$

The Exclusive-OR has both the commutative and associative properties, that is,

$$a \oplus b = b \oplus a$$

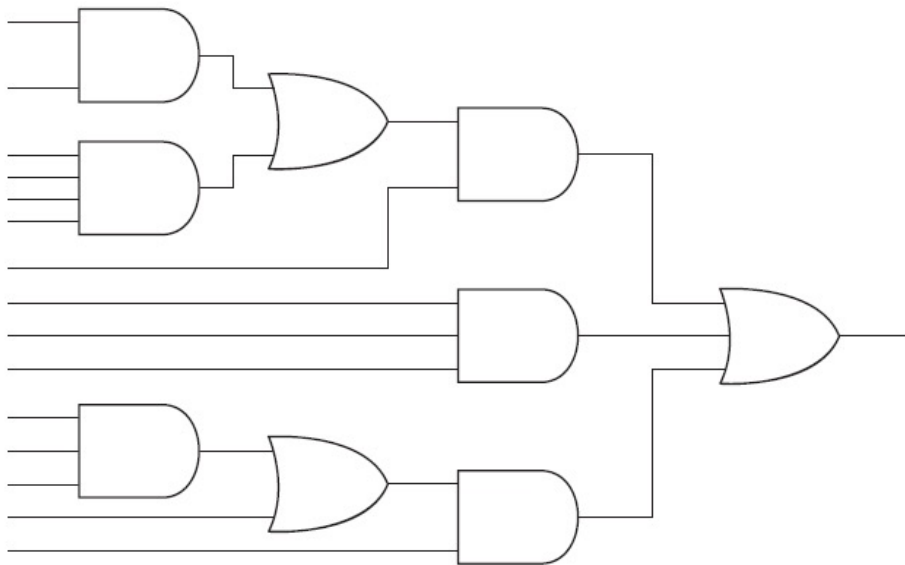
$$(a \oplus b) \oplus c = a \oplus (b \oplus c)$$

NAND, NOR, XOR IC Packages

- IC Packages:
 - 7400 4 (quadruple) two-input NAND gates
 - 7410 3 (triple) three-input NAND gates
 - 7420 2 (dual) four-input NAND gates
 - 7430 1 eight-input NAND gate
 - 7402 4 (quadruple) two-input NOR gates • 7427 3 (triple) three-input NOR gates
 - 7486 4 (quadruple) two-input Exclusive-OR gates
- To build a circuit, we utilize packages; even if we only need 1 three input NAND gate, we must buy a package with three gates on it (a 7410)
- However, a three-input gate can be used as a two-input gate by connecting two of the inputs together or by connecting one of the inputs to a logic 1

Example

Consider the following circuit, constructed with **ANDs** and **ORs**; the input variables have been omitted because they are irrelevant to the discussion.



Inputs	Gates		Packs		NAND	Packs
	AND	OR	AND	OR		
2	3	2	1		5	1
3	2	1	1	1	3	1
4	1		1		1	1
Total	6	3	3	1	9	3

Example cont.

- Number of gates and packages are shown in the left of the table
- With AND and OR gates, 4 packages are needed:
 - 3 ANDs and
 - 1 OR package (because the 2 two-input OR gates can be constructed with the leftover three-input gates)
- If all of the gates are converted to NANDs (and some of the inputs are complemented), only 3 packages are needed
- The second four-input gate on the 7420 would be used as the fifth two-input gate (by tying three of the inputs together)

Simplification of Algebraic Expressions

If a Boolean function is given in one of the two standard forms: Sum of Minterms or Product of Maxterms, following rules are commonly used

$$\text{P9a. } ab + ab' = a$$

$$\text{P9b. } (a + b)(a + b') = a$$

$$\text{P10a. } a + a'b = a + b$$

$$\text{P10b. } a(a' + b) = ab$$

although many of the other properties were used, particularly,

$$\text{P6a. } a + a = a$$

$$\text{P6b. } a \cdot a = a$$

$$\text{P8a. } a(b + c) = ab + ac$$

$$\text{P8b. } a + bc = (a + b)(a + c)$$

If the function is stated in other than one of the standard forms, two other properties are useful. First,

$$\text{P12a. } a + ab = a$$

$$\text{P12b. } a(a + b) = a$$

absorption

Simplification of Algebraic Expressions

Proof of P12a.:

$$a + ab = a.1 + a.b$$

$$= a . (1 + b)$$

$$= a.1$$

$$= a$$

Example-1

Simplify the following function: $f = xyz + x'y + x'y'$

Solution:

$$\begin{aligned} f &= xyz + x'y + x'y' = xyz + x'(y + y') = xyz + x'.1 \\ &= xyz + x' \\ &= x' + xyz \\ &= (x' + x)(x' + yz) = 1 \cdot (x' + yz) \\ &= x' + yz \end{aligned}$$

Example-2

Simplify the following functions: $g = wx + wxy + w'yz + w'y'z + w'xyz'$

Solution:

$$\begin{aligned} g &= wx + wxy + w'yz + w'y'z + w'xyz' \\ &= wx.(1+y) + w'z.(y+y') + w'xyz' && \text{(as } 1+y=1 \text{ and } y+y'=1\text{)} \\ &= wx + w'z + w'xyz' = wx + w'(z + xyz') \\ &= wx + w'(z + xy)(z + z') = wx + w'(z + xy) && \text{(as } z + z' = 1\text{)} \\ &= wx + w'z + w'xy = w'z + wx + w'xy \\ &= w'z + x.(w + w'y) = w'z + x.(w + w')(w + y) \\ &= w'z + x.(w + y) = w'z + wx + xy \end{aligned}$$

Example-3

Simplify the function: $F = (x + y)(x + y + z') + y'$

Solution:

$$\begin{aligned} F &= (x+y) (x+y+z') + y' \\ &= (x + y + 0)(x + y + z') + y' = (x + y + (0.z')) + y' \quad (\text{or use P12b}) \\ &= (x + y + 0) + y' \\ &= x + y + y' \\ &= x + 1 = 1 \end{aligned}$$

Example-4

Simplify the function: $F = (a + b' + c)(a + c')(a' + b' + c)(a + c + d)$

Solution:

$$\begin{aligned} F &= (a + b' + c)(a + c')(a' + b' + c)(a + c + d) \\ &= (a + b' + c)(a' + b' + c)(a + c')(a + c + d) \\ &= ((a.a') + (b' + c))(a + c'.(c + d)) \\ &= (0 + (b' + c))(a + (c'.c + c'.d)) \\ &= (b' + c)(a + (0 + c'.d)) = (b' + c)(a + (c'.d)) \\ &= (b' + c)(a + c')(a + d) \end{aligned}$$

Manipulation of Algebraic Functions and NAND Gate Implementations

- There is sometimes the requirement to put an expression in a certain format, such as SOP, sum of minterms, POS, or product of maxterms
- If we have SOP expression and need to expand it to sum of minterms, we have two options:
 1. Can create TT, and, from that TT, we can write the minterms against the 1s of the function, by taking primed (complemented) variables as 0 and unprimed (uncomplemented) variables as 1.
 2. Take **AND** of each term with the missing variables using property $x + x' = 1$.

Example

Consider the following function:

$$f = bc + ac + ab \quad (\text{this is in SOP form})$$

To convert it into sum of minterms form, see that terms

- 1st missing variable a , 2nd missing variable b and 3rd missing variable c .

So in order to complete the missing variables, we proceed as:

$$\begin{aligned} f &= bc.(a+a') + ac.(b+b') + ab.(c+c') \\ &= abc + a'bc + abc + ab'c + abc + abc' \\ &= 111 + 011 + 111 + 101 + 111 + 110 \\ &= m_7 + m_3 + m_7 + m_5 + m_7 + m_6 = \sum (3,5,6,7) \end{aligned}$$

Another Example

Consider the following function:

$$g = x' + xyz = x'y + x'y' + xyz \quad (\text{this is in SOP})$$

To convert it into sum of minterms form, see that terms:

- 1st missing variables 'y' and 'z', and second is not missing any variable.

So in order to complete the missing variables, we proceed as:

$$\begin{aligned} g &= x' + xyz \\ &= x'.(y+y') + xyz \\ &= x'y + x'y' + xyz \\ &= x'y.(z+z') + x'y'.(z+z') + xyz = x'yz + x'yz' + x'y'z + x'y'z' + xyz \\ &= 011 + 010 + 001 + 000 + 111 \\ &= m_3 + m_2 + m_1 + m_0 + m_7 = \sum (0, 1, 2, 3, 7) \end{aligned}$$

One more Example

Consider the following function:

$$f = (A + B + C)(A' + B') \quad (\text{this is in POS})$$

In order to convert it into product of maxterms form, see that terms:

- 1st not missing any variables while 2nd missing variable **C**.

So in order to complete the missing variable, we proceed as:

$$\begin{aligned} f &= (A + B + C)(A' + B') \\ &= (A + B + C)(A' + B' + C.C') \quad (\text{using the property } x.x' = 0) \\ &= (A + B + C)(A' + B' + C)(A' + B' + C') \quad (\text{using the distributed law of } + \text{ over } .) \\ &= 000 . 110 . 111 \quad (\text{take primed variable as 1 and unprimed variable as 0}) \\ &= M_0 . M_6 . M_7 = \pi(0, 6, 7) \end{aligned}$$

Conversion from one Form to Another

- In addition to the need to minimize algebraic expressions, there is sometimes the requirement to put an expression in a certain format, such as SOP, sum of minterms, POS, or product of maxterms
- Secondly, to meet design constraints, we sometimes must manipulate the algebra
- In order to change SOP into POS or POS to SOP, we commonly use the following rules:

$$\text{P8b.} \quad a + bc = (a + b)(a + c)$$

$$\text{P14a.} \quad ab + a'c = (a + c)(a' + b)$$

$$\text{P8a.} \quad a(b + c) = ab + ac$$

Example

Consider the following function:

$$(A + B' + C) (A + B + D) (A' + C' + D')$$

It is in POS form. In order to change it into SOP form, we use rules as:

$$\begin{aligned} & (A + B' + C)(A + B + D)(A' + C' + D') \\ &= [A + (B' + C)(B + D)](A' + C' + D') && \text{[P8b]} \\ &= (A + B'D + BC)(A' + C' + D') && \text{[P14a]} \\ &= A(C' + D') + A'(B'D + BC) && \text{[P14a]} \\ &= AC' + AD' + A'B'D + A'BC && \text{[P8a]} \end{aligned}$$

Example

Consider the following function:

$$wxy' + xyz + w'x'z'$$

It is in SOP form. In order to change it into POS form, we use rules as

$$\begin{aligned} &wxy' + xyz + w'x'z' \\ &= x(wy' + yz) + w'x'z' && \text{[P8a]} \\ &= x(y' + z)(y + w) + w'x'z' && \text{[P14a]} \\ &= (x + w'z')(x' + (y' + z)(y + w)) && \text{[P14a]} \\ &= (x + w')(x + z')(x' + y' + z)(x' + y + w) && \text{[P8b]} \end{aligned}$$

NAND NOR Implementation

Another application of P14a and this type of algebraic manipulation comes when we wish to implement functions using only two-input **NAND** or **NOR** gates (or two- and three-input gates). (We will only consider examples of NAND gate implementations.

Example

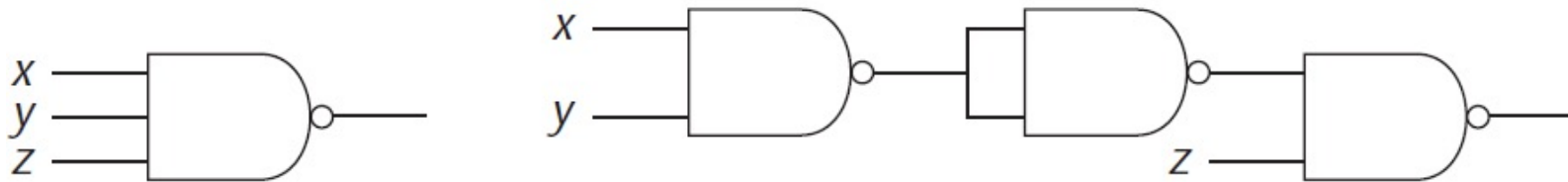
Consider the following function

$$f = ab'c' + a'c'd' + bd$$

Assume all inputs are available both uncomplemented and complemented. Find a NAND gate circuit that uses only two-input gates.

Solution

We know that a three input NAND gate can be implemented using only two-input NAND gates as:



Example cont.

$$f = ab'c' + a'c'd' + bd$$

- Two-level solution would require 4 gates, three of which would be three-input gates and 11 gate inputs
- To solve this problem, we must eliminate three-input gates
- Starting point is to attempt to factor something from three literal terms
- c' is a common in first two terms, and we can thus obtain

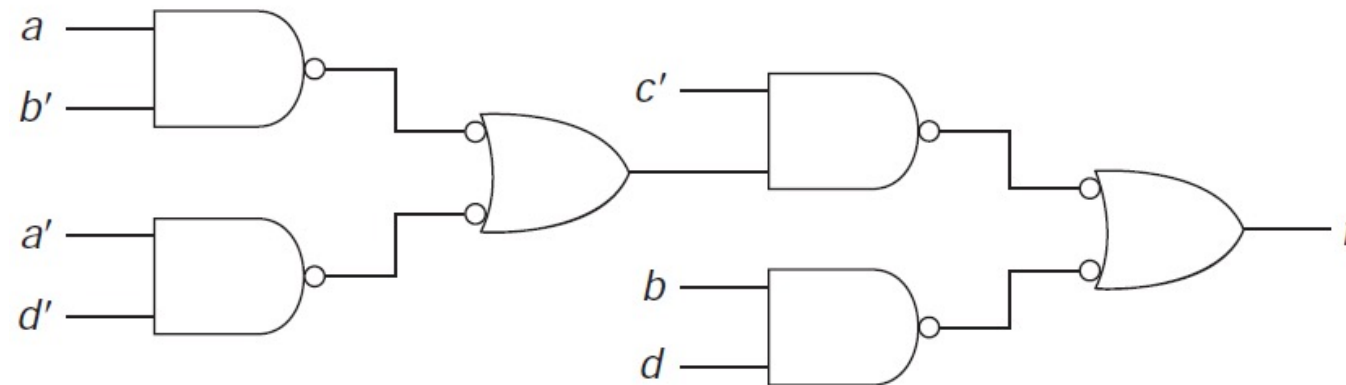
$$f = c'(ab' + a'd') + bd$$

Example cont.

This function can be implemented using only 2-input NAND gates as:

$$f = c'(ab' + a'd') + bd$$

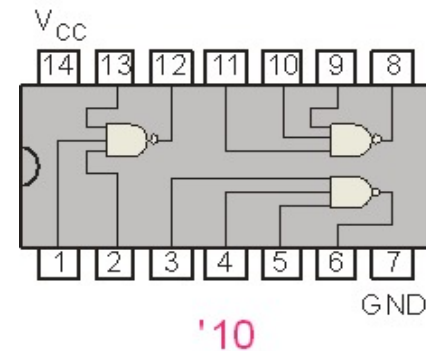
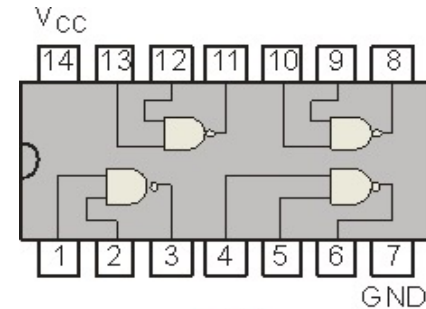
Figure 2.22 A two-input NAND gate circuit.



Solution requires 6 gates and 12 inputs

Example cont.

- Either solution, requires 2 IC packages – Two-level solution would require
 - 7410 (3 three input gates) and
 - 7400 for the remaining two-input gate and would leave 3
 - of those gates unused – Second solution
 - Two 7400s (4 two-input NANDs each) and would leave two of the gates unused
- If we had replaced each three-input gate by 2 two-input ones plus a NOT, the implementation would require 7 two input gates plus three NOT gates





The End